
ADVERSARIAL SEARCH AND GAMES

INSTRUCTOR: MOAYAD ALNAMMI

malnammi.github.io

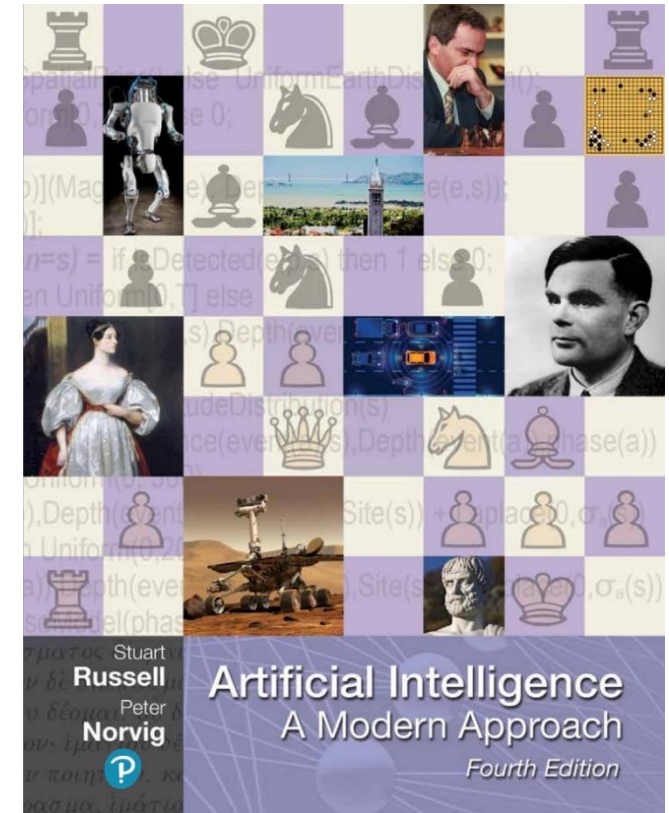
Notice

- Slides prepared by Moayad Alnammi – malnammi.github.io
- Some figures adapted from CS188: Introduction to Artificial Intelligence, UC Berkeley (Dan Klein, Pieter Abbeel) — ai.berkeley.edu
- Some figures are obtained from the [AIMA Book](#)
- Some content adapted from Dr. ElSayed El-Alfy
- Some content adapted from Dr. Richard Lathrop (UC Irvine)

This deck is published at malnammi.github.io under CC BY-NC-SA 4.0. Reuse and adaptation require attribution to the author and the sources above; commercial use is not permitted; adaptations must carry the same license.

Resources & Reading

- Textbook website: <http://aima.eecs.berkeley.edu/>
- Read Chapter 6 in AIMI 4/E



Outline

- Games typical assumptions
- Games vs. search problems
- Types of games
- Game search tree
- Adversarial search algorithms
 - Minimax
 - Alpha-beta pruning
 - Cutoff search using heuristics or estimates
 - Expectation based adversarial searches
- Summary

Adversarial Games

Adversarial games – Intro

- In real life, there are many competitive environments in which there are two or more agents **competing** against each other. E.g. sports, markets, combat, etc.
- Here we'll discuss simple two-player games like chess where the actions and rules are well defined. These are suitable for AI education and research.
- Games can be:
 - Deterministic or stochastic. Stochastic games have element of randomness involved like dice throwing or shuffling of cards.
 - Perfect information or imperfect information (fully-observable vs partially observable).
 - Zero-sum or not. Zero-sum games in a two-player setting means that if one agent gains then the other agent must lose an equal amount. The sum of all player rewards is 0.

Game examples used for AI research		Deterministic	Stochastic
Perfect Information		Chess, Checkers, Go	Backgammon, Monopoly
Imperfect information		Blind tictactoe, blind chess, Battleships,	Bridge, Scrabble

Search problems vs games

- Search
 - Not adversarial. Single agent operating in an environment.
 - Solution is a path to the goal; or finding a goal state configuration.
 - Finds optimal solution.
 - Evaluation/heuristic function: $f(n)$ estimates cost from s to g through n .
 - Applications: path planning, scheduling activities, CSPs, finding optimal configuration.
- Games
 - Adversarial. Involves multiple agents operating against each other.
 - Solution is a **strategy (policy)** that specifies a move for every possible adversary reply.
 - May have time limits, cannot search for optimal action, instead finds *approximately* optimal action.
 - Evaluation function: evaluates how good a game state is for the agent to achieve a win.
 - Applications: chess, checkers, Othello/Go, backgammon, multi-agent environments.

Game Assumptions

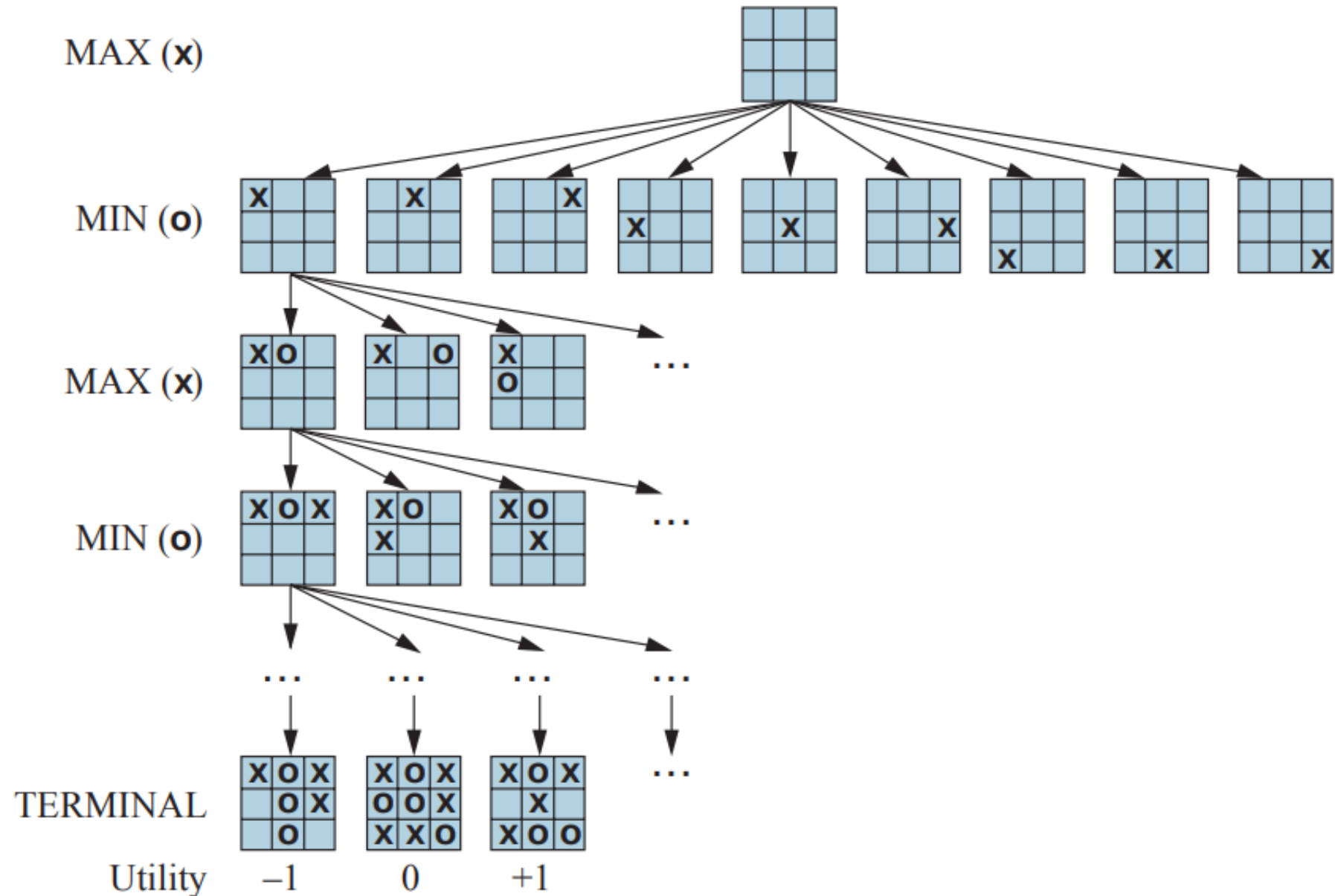
- As a start, we will discuss games with the following assumptions:
 - Two agents whose actions alternate (turn-taking). An action taken by an agent is called a **ply** or **turn**. So, a 4-ply (or 4-turn) means that 4 actions were taken (one by agent A, then agent B, then A, then B).
 - Deterministic & fully-observable environments. This is an important consideration; this means we can *simulate* the game states computationally.
 - Zero-sum games: **Utility (reward)** values for each agent are the opposite of the other (competitive agents). The sum of all player utility (rewards) is 0.

$$\sum_{p \in Players} utility(p) = 0$$

- Techniques applied here can generalize to stochastic games, multiple players, non-zero sum, etc. with modifications to account for the setting.

Example: Tic-tac-toe Game-tree

- We refer to the two players (agents) MAX and MIN.
- MAX moves first and then the two agents take turns until game is over. Once the game is over, points and penalties are awarded to the two players.
- The tree shown is the full game-tree for tictactoe starting from an empty board.
- Each level is a player's turn.
- Leaves are end-game states.

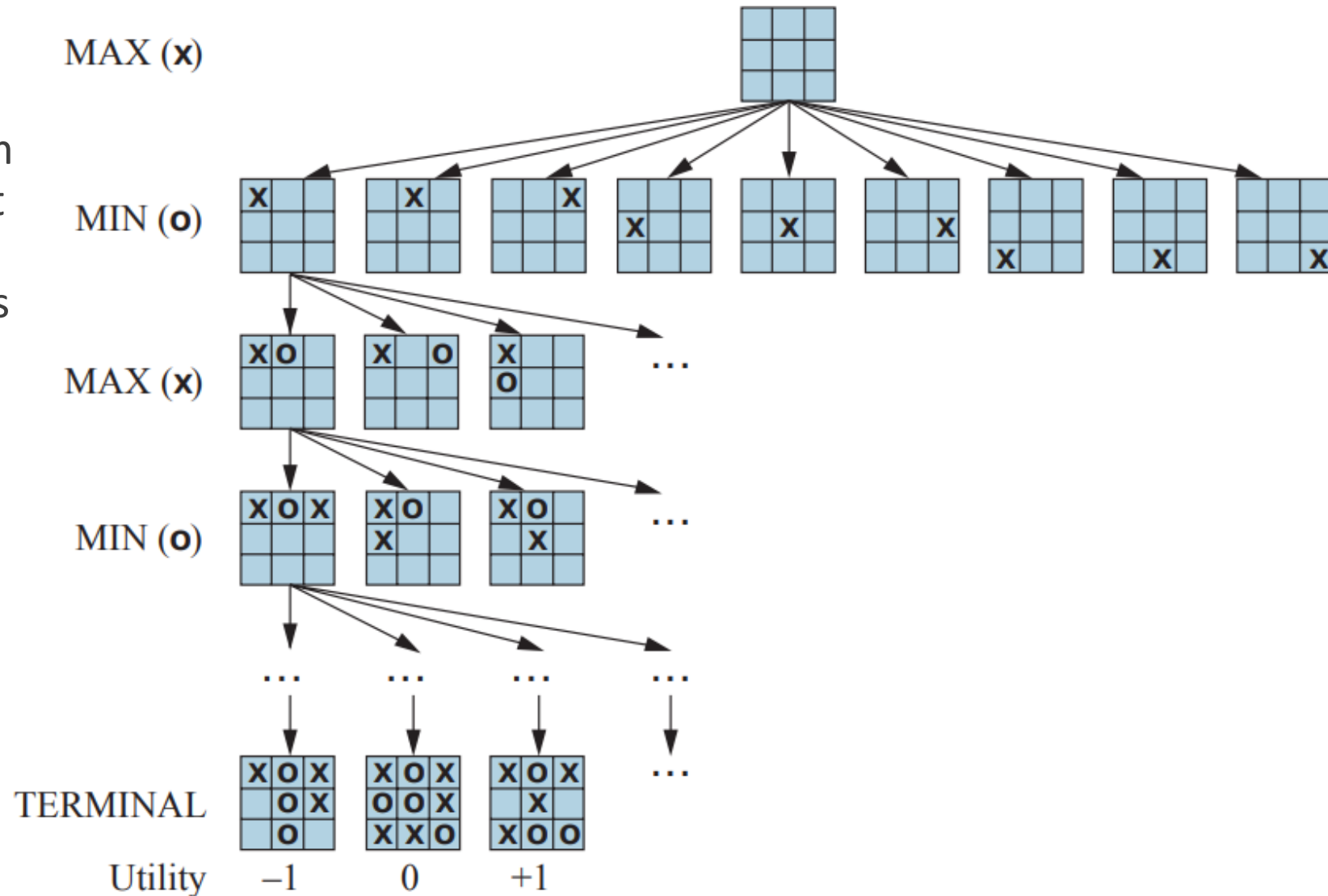


Two-player zero-sum game setup

- We refer to the two players (agents) MAX and MIN.
- MAX moves first and then the two agents take turns until game is over. Once the game is over, points and penalties are awarded to the two players.
- Formally, we can define a game as follows:
 - S_0 the initial state of the game.
 - *To-Move(s)*: specifies the player whose turn it is to move in state s .
 - *Actions(s)*: specifies the actions (moves) that are allowed in state s .
 - *Result(s, a)*: the state-transition model that specifies the state resulting from applying action a in state s .
 - *Is-Terminal(s)*: true/false specifying if the game has ended (i.e. terminal state).
 - *Utility(s, p)*: A utility function specifying the numeric value given to player p in terminal state s . For example in chess, we can award +1 (winner), -1 (loser), and 0 (draw). In backgammon, can range from 0 to 192.
- Given the current state of the game, MAX **computationally** searches a game-tree to select the next move. Think of MAX as the agent we are designing to beat its opponent MIN.

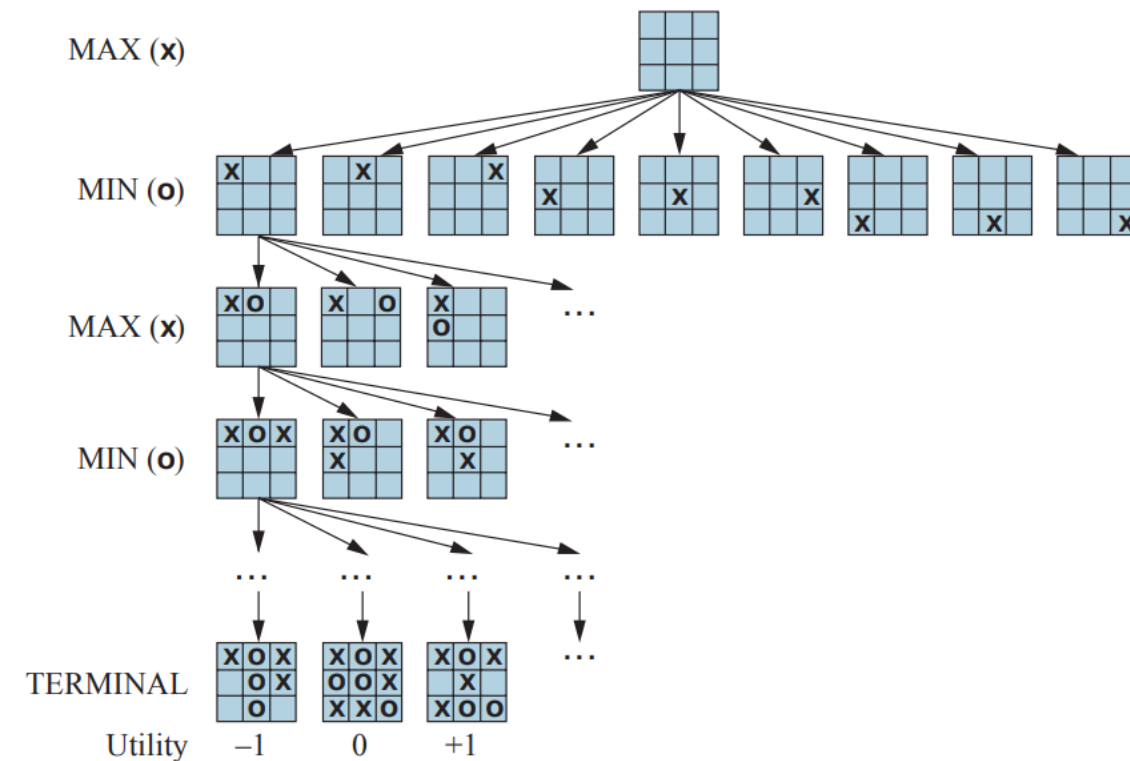
Example: Tic-tac-toe Game-tree

- Here the game-tree is starting from the empty board of tic-tac-toe. But note that the game-tree can start from whatever the current board is in real life.
- In tic-tac-toe there are a total of $9! = 362,880$ terminal nodes.
- For Chess there are over 10^{40} .
- Takeaway: we cannot realize the full game-tree for most games.



Optimal decision-making in games (algorithms)

- Given the current state, MAX **computationally** searches a game-tree to select the next move.
- minimax algorithm**: MAX searches the **entire** game-tree, then selects the **optimal** action at the root.
- Optimizations that can be done to speed up search:
 - **alpha-beta pruning**: MAX can maintain some extra variables and avoid searching up to **half** of the tree, while still selecting the **optimal** action.
 - **Heuristics**: MAX can use techniques to speedup action selection like using **heuristic evaluation functions** to judge how good a state is. With this, MAX does not have to realize the full tree to make a decision. **Not guaranteed to be optimal**.
 - **Simulation/Monte-Carlo**: MAX can *simulate* the game from a node in the game-tree, then gather statistics of win/loss ratios from the simulation. Select action that give good statistics. **Not guaranteed to be optimal; however converges to optimality with more simulations (more accurate stats)**.



Optimal decision-making in games

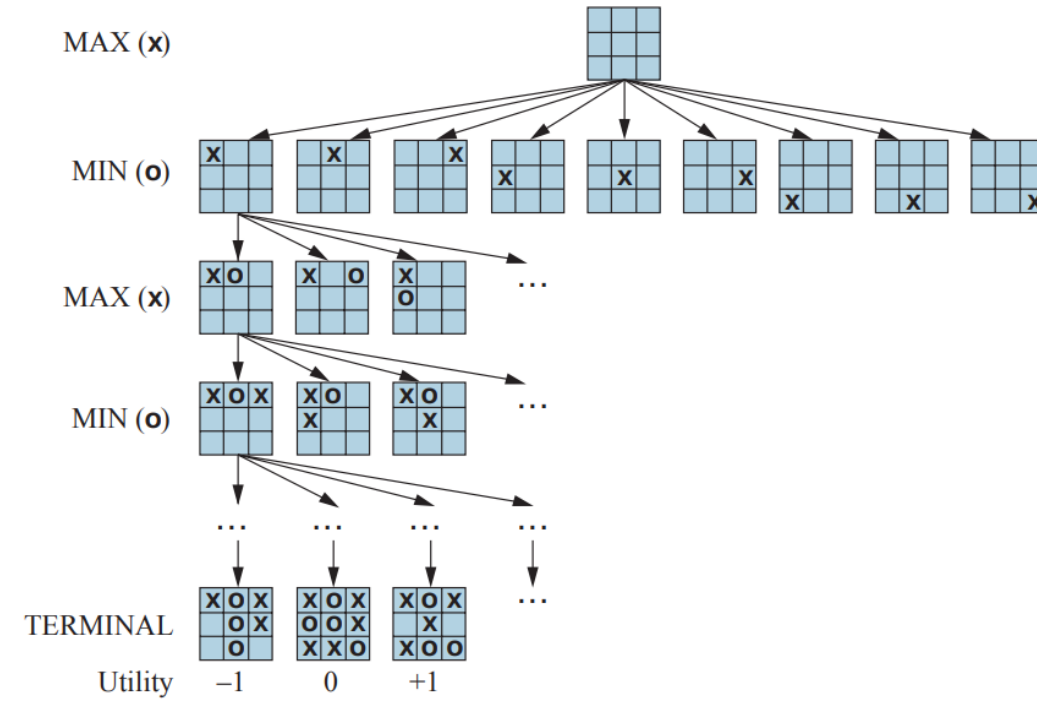
Minimax algorithm

Optimal decision-making: minimax search [1]

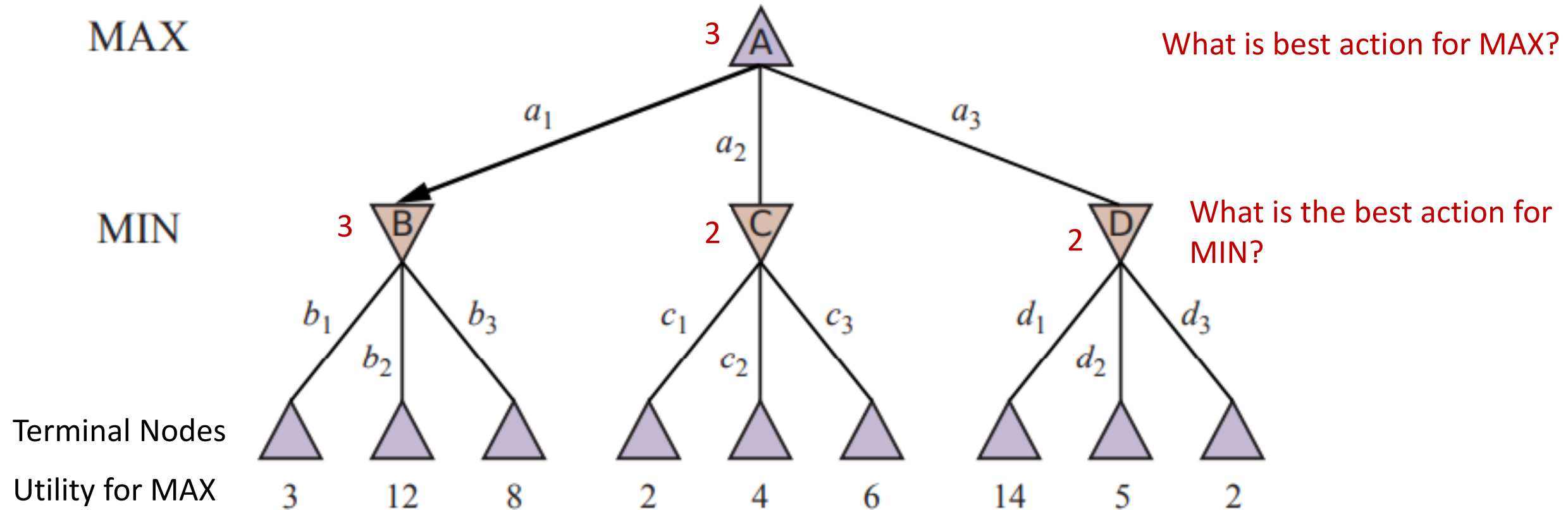
- For most games, MAX cannot hope to realize the full game-tree.
- But let us assume that MAX can realize and search the full game-tree.
- If MAX can realize and search the full game-tree, what algorithm can be employed to make the optimal action?
- First, we should ask: how will MAX and MIN play?
 - Reasonable assumption is that both play optimally (i.e. maximizing their utility).
- **Minimax search:** choose the action at the root that gives a game-state with highest **true** utility value.

Syntax & Semantics of the utility for the two-player game-tree

- MAX is the agent we are designing. We want to design an agent that will maximize its own utility.
- MIN is the opponent.
- The assumption is that MAX and MIN play optimally.
- The game-tree terminal nodes are showing the utility for MAX perspective.
- Since it is zero-sum game, a negative utility for MAX, indicates a positive utility for MIN.
- Thus, MIN is trying to minimize MAX's utility which will mean increasing its own utility.
- This is just notation/format used in book. This means that MAX will take maximizing actions, whereas MIN will take minimizing actions. We could have had both take maximizing action if we used $Utility(s, player)$ where utility is based on the player perspective.



- **Minimax search:** move to the state with highest **true** utility value.
- Below is the game-tree for a 2-ply game (only 2 turns and game ends).
- Recall that utility is awarded for winner and loser at the terminal nodes (leaves). Here we show utility for MAX perspective, but since this is zero-sum game, MIN is gaining when MAX loses utility.



- **Minimax search:** move to the state with highest **true** utility value.
- So, the algorithm computes a minimax value for a state s , in a recursive manner.
- Thus, the implementation can be implemented recursively.
- Note this is essentially doing tree-like DFS on the game-tree.

$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if } \text{IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{To-Move}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{To-Move}(s) = \text{MIN} \end{cases}$$

function MINIMAX-SEARCH(*game, state*) **returns** *an action*

player $\leftarrow$ *game*.TO-MOVE(*state*)

value, move $\leftarrow$ MAX-VALUE(*game, state*)

return *move*

function MAX-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state, player*), *null*

v, move $\leftarrow -\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, a2 $\leftarrow$ MIN-VALUE(*game, game*.RESULT(*state, a*))

if *v2* > *v* **then**

v, move $\leftarrow$ *v2, a*

return *v, move*

Function represents MAX-node

If terminal reached, eval position

Otherwise, find best child; notice we are doing maximization.

function MIN-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state, player*), *null*

v, move $\leftarrow +\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

v2, a2 $\leftarrow$ MAX-VALUE(*game, game*.RESULT(*state, a*))

if *v2* < *v* **then**

v, move $\leftarrow$ *v2, a*

return *v, move*

Function represents MIN-Node

If terminal reached, eval position

Otherwise, find best child; notice we are doing minimization.

Minimax properties

- Optimality?
 - We made an assumption that MIN plays optimally as well.
 - What about if MIN plays sub-optimally? Then MAX will achieve utility at least as good as when MIN plays optimally. In other words, MAX **may** achieve better performance if MIN plays sub-optimally.
- Minimax is similar to **tree-like DFS**
 - Let's say that game ends in $O(d)$ turns, and the branching factor is $O(b)$.
 - Time complexity? $O(b^d)$
 - Space complexity? $O(bd)$? With recursive implementation it could be $O(d)$ depending on how we generate the actions list from `game.actions()`.
 - Complete? Yes, if game-tree is finite (game has end conditions).
- Notes:
 - In the given pseudocode, the minimax algorithm is given a game-state and returns the best action that MAX should take. This is all done computationally by MAX.
 - After computing the best decision, MAX can then take that decision in the real-world.
 - The opponent MIN will then take an action.
 - Then MAX can computationally run minimax algorithm from the current game-state. And so on.

Game-tree size discussion

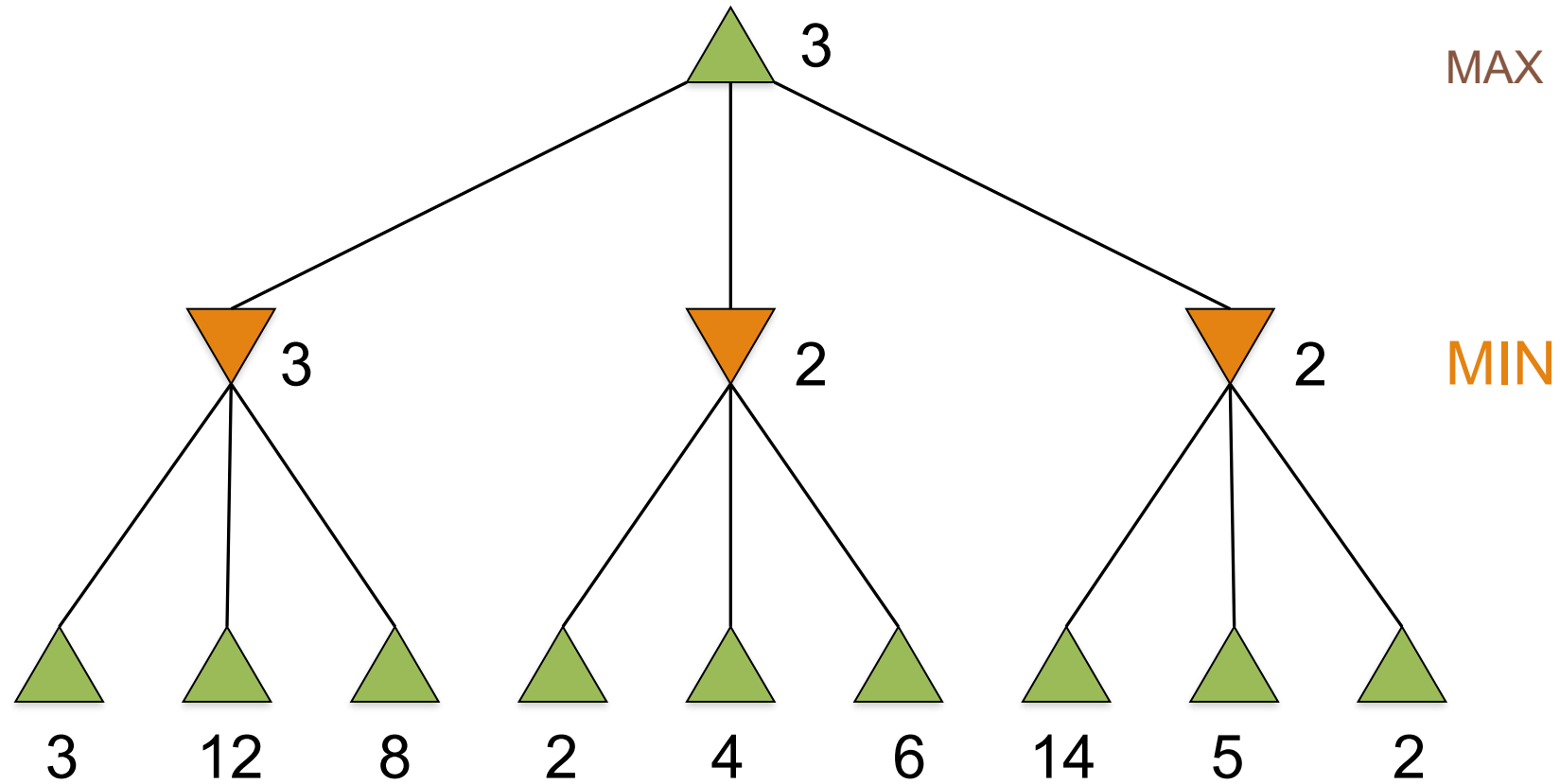
- **Tic-tac-toe**
 - $b \approx 5$ legal actions per state on average; total 9 plies (turns) in game
 - Game-tree size $5^9 = 1,953,125$ game-tree nodes
 - Exact solution is quite reasonable on modern machines
- **Chess**
 - $b \approx 35$ (approximate average branching factor)
 - $d \approx 100$ (depth of game tree for typical game)
 - Game-tree size $b^d = 35^{100} \approx 10^{154}$ game-tree nodes
 - Exact solution infeasible
- For many games, exact solution with minimax usually infeasible.
- But, do you need to explore the whole game-tree?
 - Idea 1: (**alpha-beta pruning** idea).
- Game-playing emphasizes being able to make best decisions in a finite amount of time.
 - Idea 2: **cutoff** search and *estimate* utility using **heuristic evaluation** function
 - Idea 3: cutoff search and *estimate* utility using **simulation statistics**.

Minimax with Alpha-beta Pruning

Minimax with alpha-beta pruning

- Minimax can compute the optimal action, but requires processing the entire game-tree.
- However, we can prune large parts of the game-tree that make no difference on the final outcome. That is, we can compute the optimal action without searching the entire game-tree.
- **Alpha-beta pruning** is such a technique.
- Idea: *each* node in the game-tree keeps track of two values
 - α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
 - β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- Thus, at any node in the game-tree we know that $\alpha \leq \text{bestvalue} \leq \beta$

Minimax without alpha-beta pruning



Minimax **with** alpha-beta pruning [1]

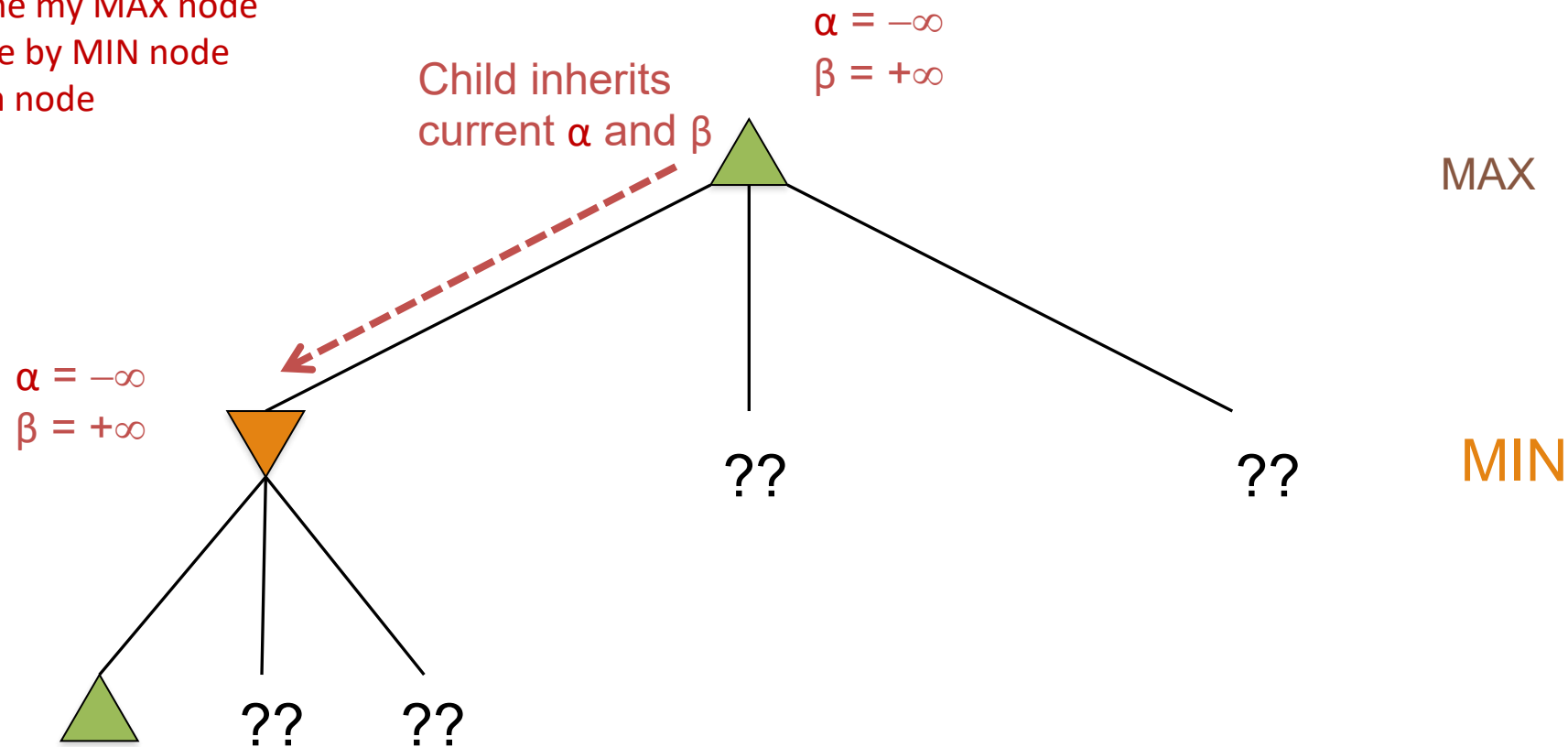
- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- Initial best-values at root are unknown so set ($\alpha = -\infty$, $\beta = +\infty$); do DFS to terminal leaf node.

Prune when $\alpha \geq \beta$

$\alpha = \text{MAX}(\alpha, v)$ only done by MAX node

$\beta = \text{MIN}(\beta, v)$ only done by MIN node

Update bestval of each node



Minimax **with** alpha-beta pruning [2]

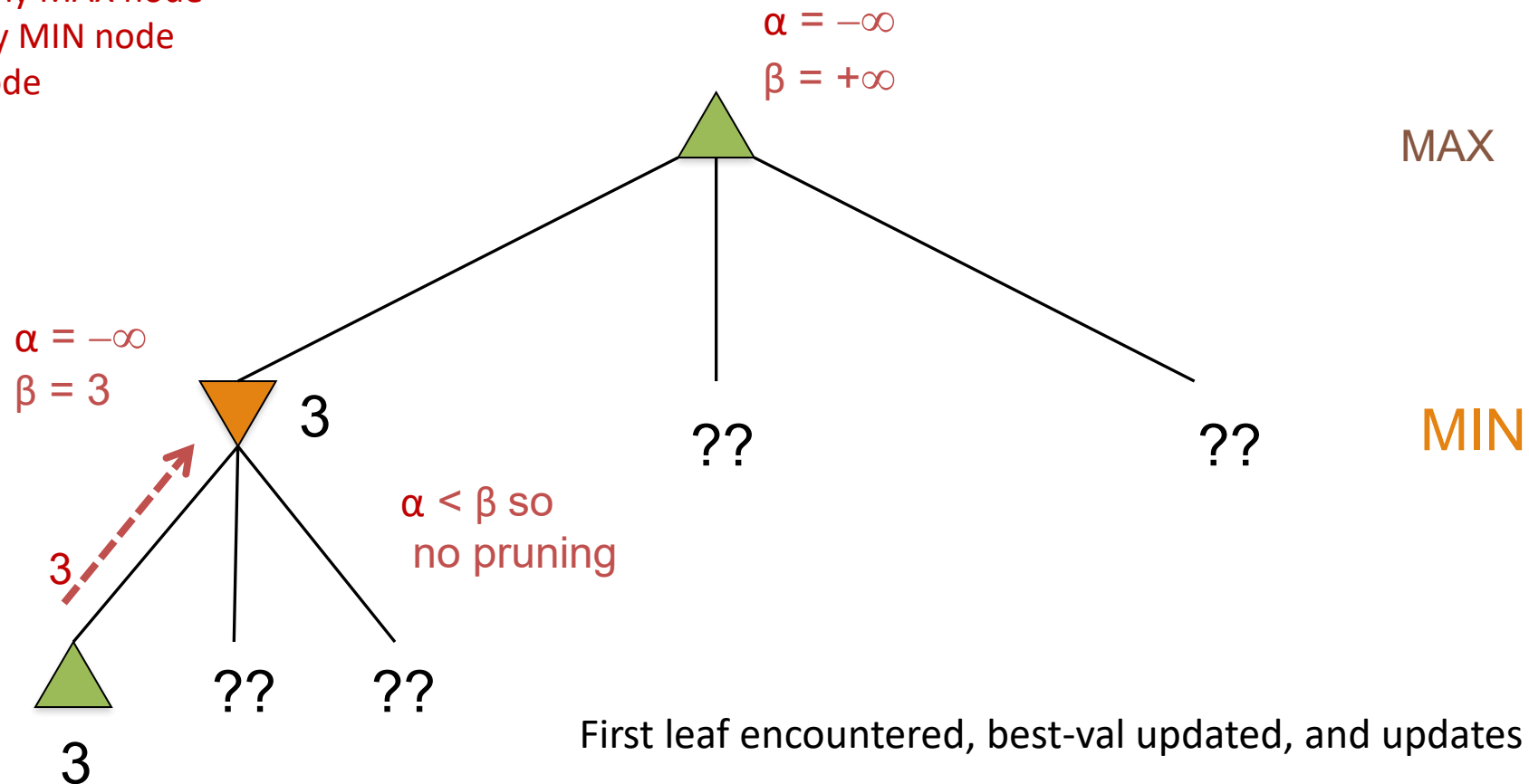
- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.

Prune when $\alpha \geq \beta$

$\alpha = \text{MAX}(\alpha, v)$ only done by MAX node

$\beta = \text{MIN}(\beta, v)$ only done by MIN node

Update bestval of each node



Minimax **with** alpha-beta pruning [3]

- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- See remaining leaves. Then pass alpha-beta values to caller; MAX updates α

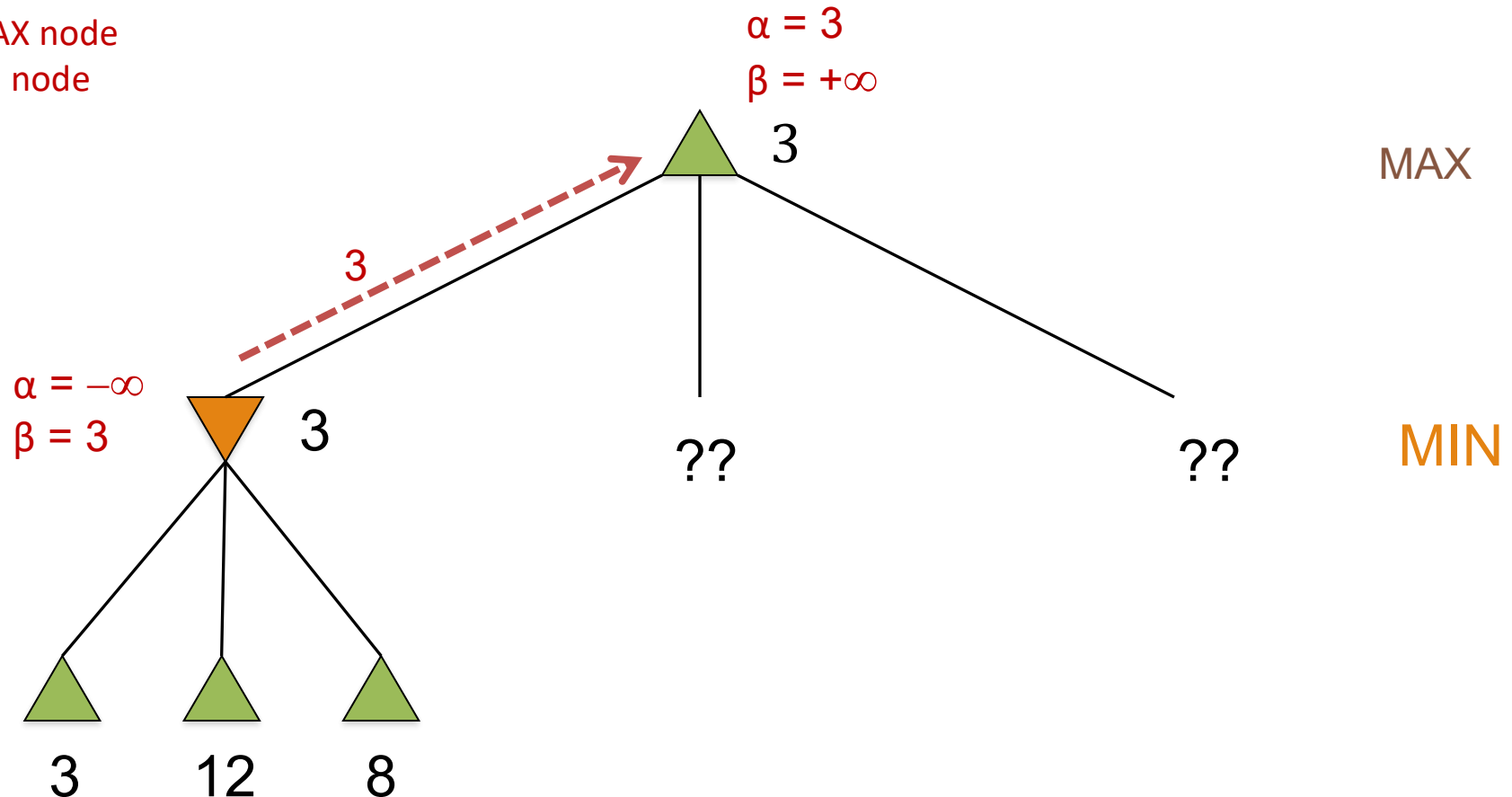
Prune when $\alpha \geq \beta$

$\alpha = \text{MAX}(\alpha, v)$ only done by MAX node

$\beta = \text{MIN}(\beta, v)$ only done by MIN node

Update bestval of each node

Check remaining
leaves; none change
the best-val or β



Minimax **with** alpha-beta pruning [4]

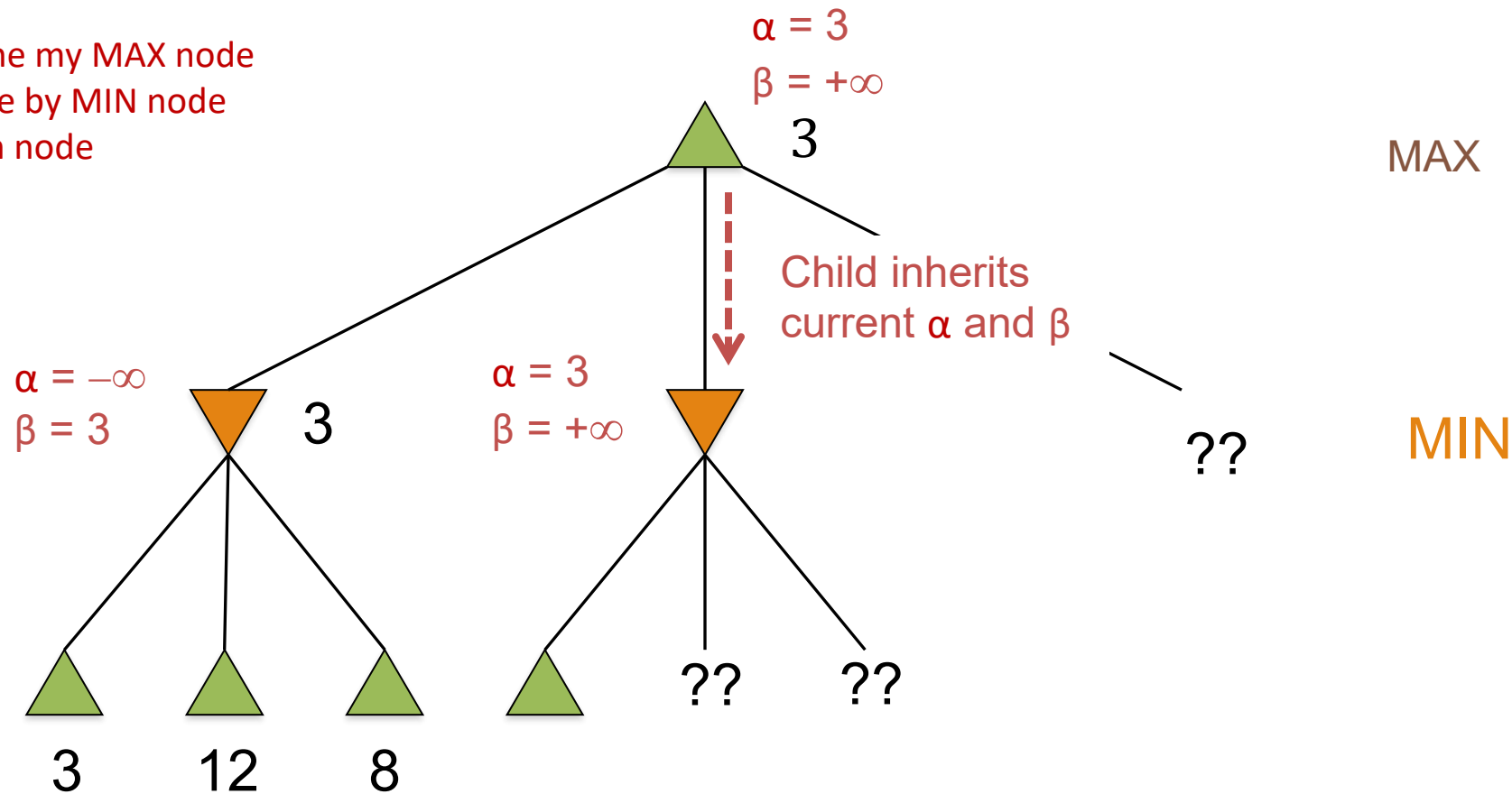
- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- Pass α , β to descendants

Prune when $\alpha \geq \beta$

$\alpha = \text{MAX}(\alpha, v)$ only done by MAX node

$\beta = \text{MIN}(\beta, v)$ only done by MIN node

Update bestval of each node



Minimax **with** alpha-beta pruning [5]

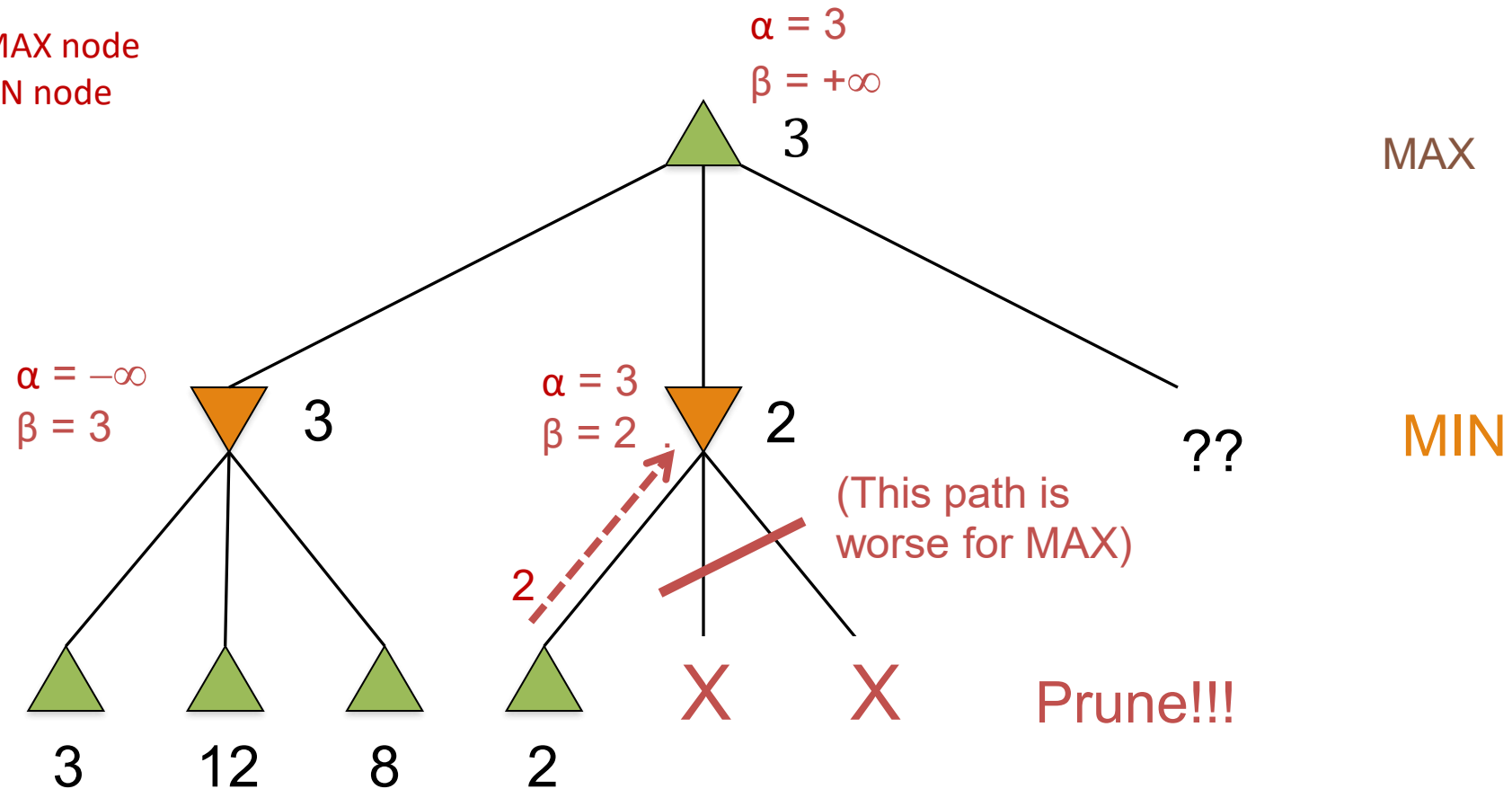
- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- Observe leaf value; MIN's level; MIN updates β . Prune appropriately.

Prune when $\alpha \geq \beta$

$\alpha = \text{MAX}(\alpha, v)$ only done by MAX node

$\beta = \text{MIN}(\beta, v)$ only done by MIN node

Update bestval of each node



Minimax **with** alpha-beta pruning [6]

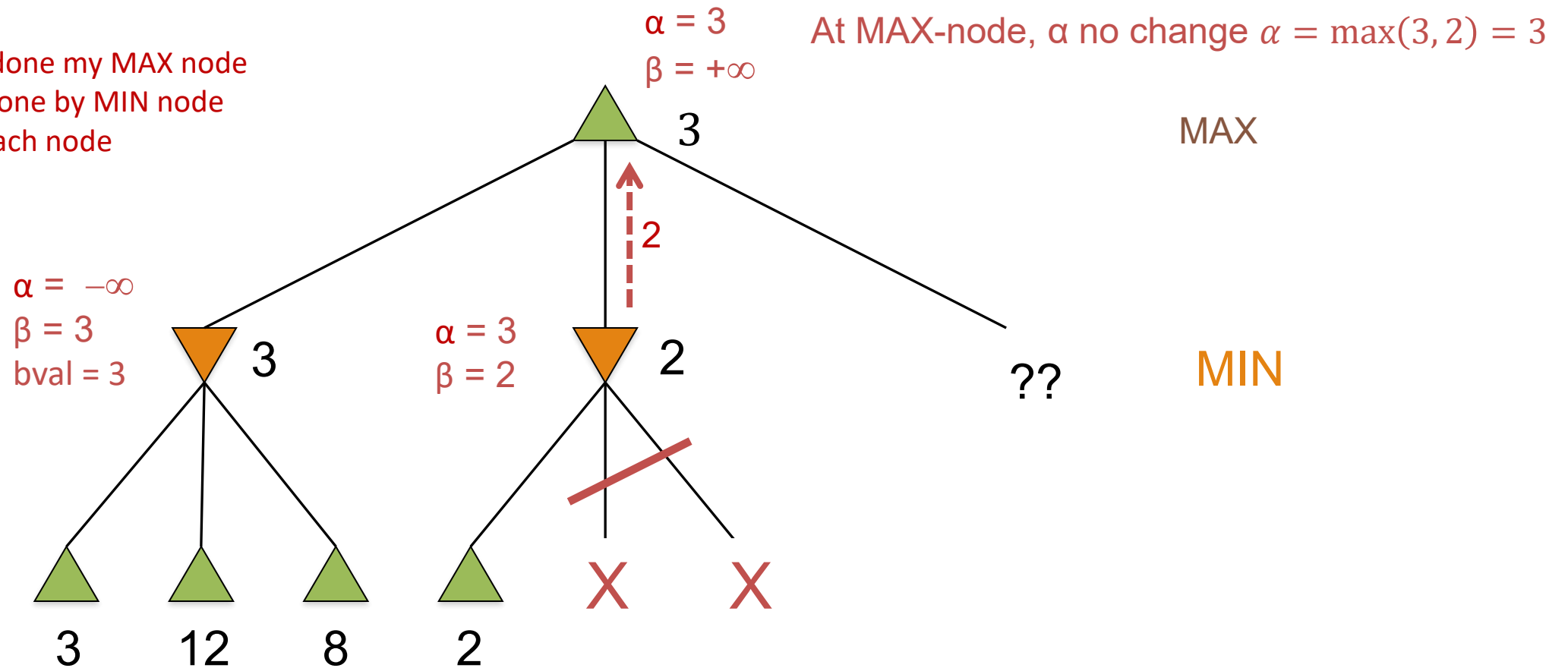
- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- Pass outcome to caller & update caller.

Prune when $\alpha \geq \beta$

$\alpha = \text{MAX}(\alpha, v)$ only done by MAX node

$\beta = \text{MIN}(\beta, v)$ only done by MIN node

Update bestval of each node



Minimax **with** alpha-beta pruning [7]

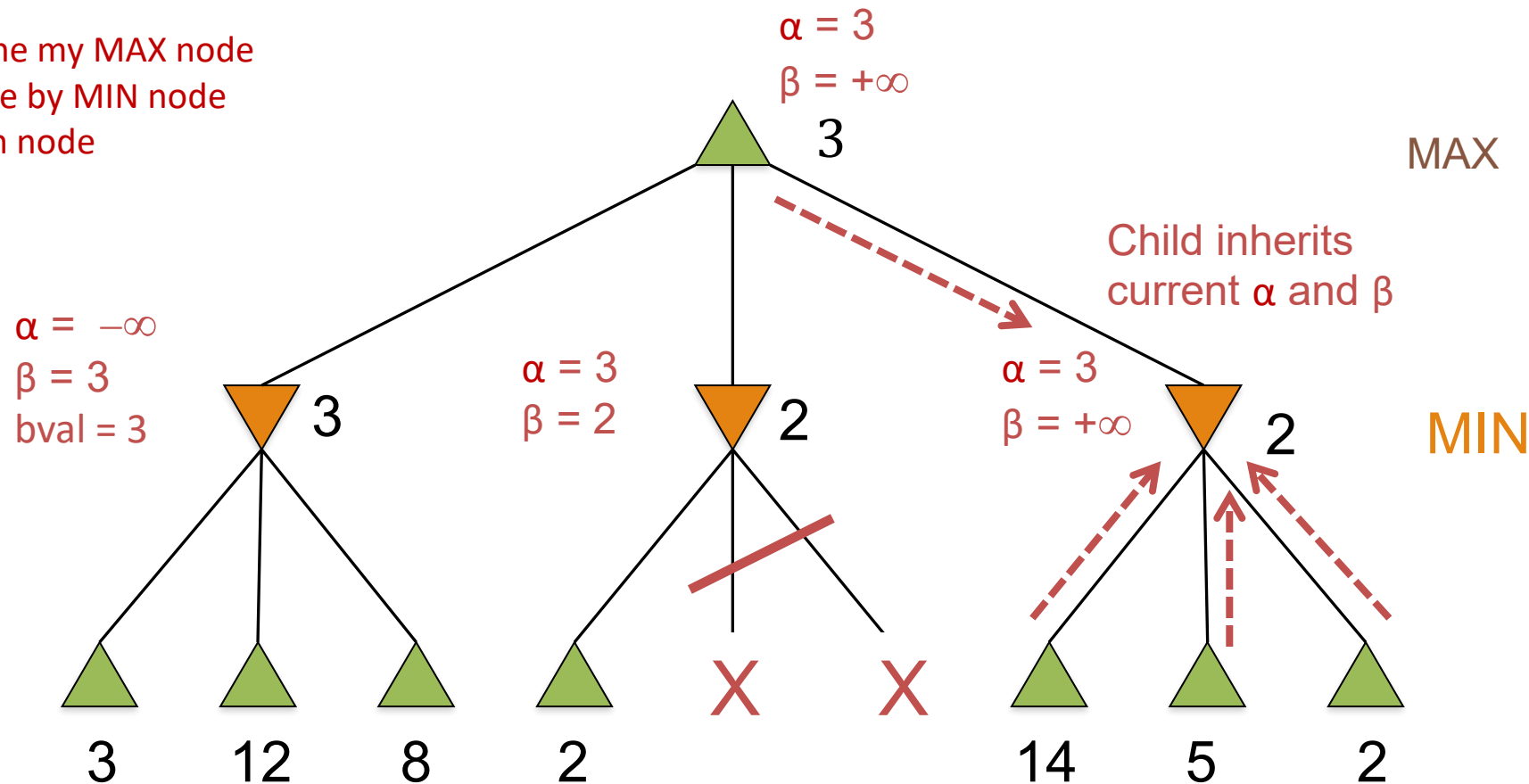
- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- Continue DFS at right child of root. No pruning at right subtree.

Prune when $\alpha \geq \beta$

$\alpha = \text{MAX}(\alpha, v)$ only done by MAX node

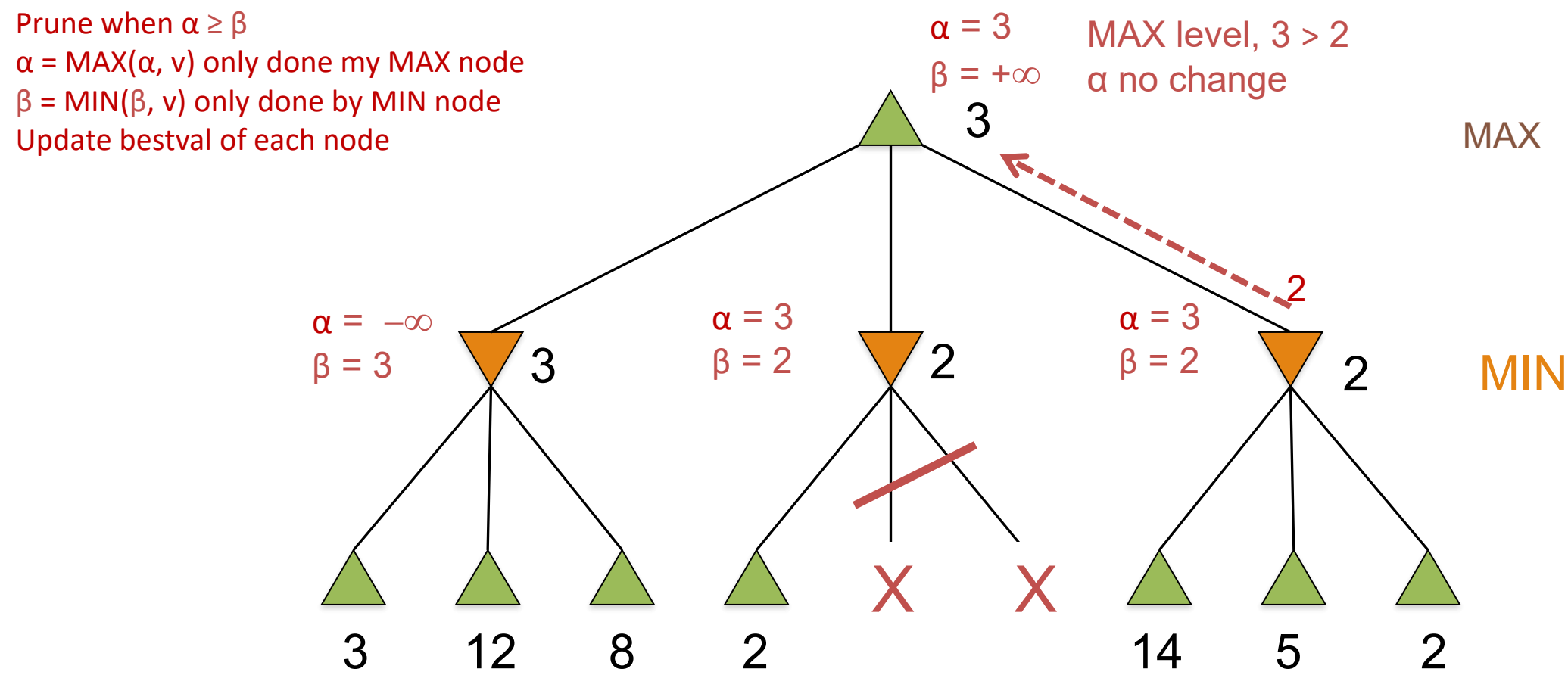
$\beta = \text{MIN}(\beta, v)$ only done by MIN node

Update bestval of each node



Minimax **with** alpha-beta pruning [8]

- α : lower-bound on the best utility value that MAX can achieve. Only MAX-nodes can change this value.
- β : upper-bound on the best utility value that MAX can achieve. Only MIN-nodes can change this value.
- Pass outcome to caller & update caller. Value at root is returned as max of children.



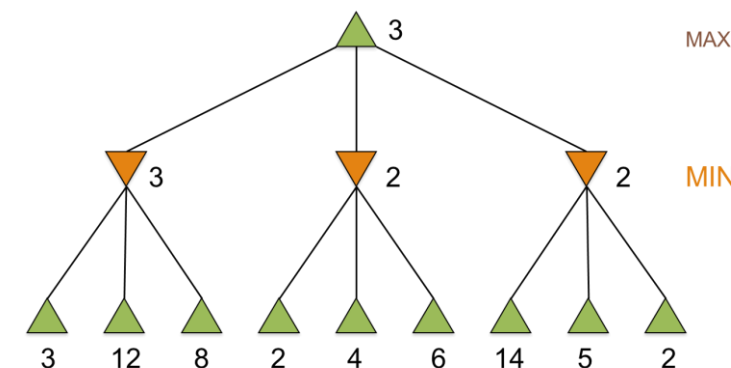
Minimax with alpha-beta pruning – algorithm

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action
 $\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$
 $\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state}, -\infty, +\infty)$
 return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), null
 $v \leftarrow -\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 > v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 if $v \geq \beta$ **then return** *v*, *move*
 return *v*, *move*

function MIN-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if *game.IS-TERMINAL*(*state*) **then return** *game.UTILITY*(*state*, *player*), null
 $v \leftarrow +\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MAX-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 < v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\beta \leftarrow \text{MIN}(\beta, v)$
 if $v \leq \alpha$ **then return** *v*, *move*
 return *v*, *move*

- α : lower-bound on the best utility value that MAX can achieve.
- β : upper-bound on the best utility value that MAX can achieve.
- $\alpha \leq \text{bestvalue} \leq \beta$
- **Prune when $\alpha \geq \beta$**



Only MAX nodes change alpha

At MAX-node, search is cut when value of node is bigger than upper-bound; because the parent MIN-node above would never select this action as it would give **higher** value.

Only MIN nodes change beta

At MIN-node, search is cut when value of node is less than lower-bound; because the parent MAX-node above would never select this action as it would give **lower** value.

Efficiency of alpha-beta pruning

- Despite pruning parts of the game-tree, can still guarantee optimal decision.
- Good **child ordering** improves effectiveness of pruning.
- Space complexity: Since we are still using tree-like DFS type algorithm, it is $O(bd)$ or $O(d)$ depending on exact implementation. Either way, linear in the game parameters.
- Worst-case time-complexity:
 - Branch exploration are ordered so that no pruning takes place. In this case alpha-beta gives no improvement over exhaustive minimax search. Recall minimax is $O(b^d)$
- Best-case time-complexity with “perfect ordering”:
 - Each player’s best move is the left-most alternative (i.e., evaluated first)
 - Time complexity is $O(b^{d/2})$
 - Can solve a tree roughly twice as deep as minimax.
- With random ordering: *average* time complexity is around $b^{3d/4}$
- In practice, can use simple/heuristic-based ordering schemes to get close to $O(b^{d/2})$
- Even with perfect ordering, $O(b^{d/2})$ is still too large for games like chess and Go. Too many states to explore.
- Get more practice here: <https://pascscha.ch/info2/abTreePractice/>

Limited resources – speeding up the
game search with **heuristics**

Heuristics to speed up the search.

- Even with perfect ordering, $O(b^{d/2})$ is still too large for games like chess and Go. Too many states to explore.
- Furthermore, some games have time-limits for an action, i.e. 1 minute to make a move.
- What can we do to speed up the search?
- We can **cutoff** the search at certain nodes in the game-tree by applying a **heuristic evaluation function**, thereby treating non-terminal nodes as terminal nodes.
- The cutoff condition can be anything; e.g. depth-limit, or states with certain conditions.
- Let us first apply this idea to the minimax algorithm.

Minimax with heuristics: H-Minimax [1]

$\text{MINIMAX}(s) =$

$$\begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if } \text{IS-TERMINAL}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{To-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{To-MOVE}(s) = \text{MIN} \end{cases}$$

- We replace $\text{UTILITY}(s, p)$ with the **heuristic** $\text{EVAL}(s, p)$ which is an **estimate** of the expected utility of state s for player p .
- We also replace the terminal test with a cutoff test. This should also return true for terminal nodes.
- Note that we also pass the depth d with recursive calls.

$\text{H-MINIMAX}(s, d) =$

$$\begin{cases} \text{EVAL}(s, \text{MAX}) & \text{if } \text{CUTOFF-TEST}(s, d) \\ \max_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if } \text{To-MOVE}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{H-MINIMAX}(\text{RESULT}(s, a), d + 1) & \text{if } \text{To-MOVE}(s) = \text{MIN}. \end{cases}$$

Heuristic/Evaluation function properties

- Heuristic/evaluation function:
 - Estimates how good the current game-state configuration is for a player.
 - Typically, evaluate how good it is for the player p , and how good it is for the opponent, and subtract the opponent's score from the player's score.
 - Ex: Chess: Value of all white pieces - Value of all black pieces
- The heuristic/evaluation function (EVAL) must-have properties:
 - For terminal states: $EVAL(s, p) = UTILITY(s, p)$
 - For non-terminal states, heuristic value should be between win/loss utility values of the game. Formally: $UTILITY(loss, p) \leq EVAL(s, p) \leq UTILITY(win, p)$
- Other than these two requirements, what other properties are good?
 - Not take too long to compute. The point of using heuristics is to speedup the search.
 - Should correlate with the chances of winning. Otherwise, it would be unhelpful.
 - For non-terminal states s and s' , if state s more likely leads to wins than state s' , then: $EVAL(s, p) > EVAL(s', p)$

Developing a heuristic function

- How to develop a heuristic?
- One idea is to compute *features* from a given state, then the heuristic is a **function of these features**.
- For example, in a chess state we can use a **linear weighting** of the following features:
 - $f_1(s) = (\text{num white pawns} - \text{num black pawns})$
 - $f_2(s) = (\text{num white knights} - \text{num black knights})$
 - $f_3(s) = (\text{num white queens} - \text{num black queens})$
 - Thus, $\text{EVAL}(s, \text{MAX}) = w_1 f_1(s) + w_2 f_2(s) + w_3 f_3(s)$
- The linear weighting is assuming that the features of a state are **independent** of each other.
- In some problems, it may be better to combine features **non-linearly**.
 - E.g. $\text{EVAL}(s, \text{MAX}) = w_1 f_1(s) f_2(s) + w_3 f_3(s)^{f_1(s)}$
- A question arises if we develop heuristics in this way. How do we set the weights?
 - Run computational experiments with various weights and compare the results.
 - Better idea. Machine learning: we use experience from previous games (historical data), where the data is (state, outcome) pairs.

function MINIMAX-SEARCH(*game*, *state*) **returns** an action

player $\leftarrow$ game.TO-MOVE(*state*)

value, *move* $\leftarrow$ MAX-VALUE(*game*, *state*)

return *move*

How to modify this minimax pseudocode with heuristic?

function MAX-VALUE(*game*, *state*) **returns** a (*utility*, *move*) pair

if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, *player*), null

v, *move* $\leftarrow -\infty$

for each *a* **in** game.ACTIONS(*state*) **do**

v2, *a2* $\leftarrow$ MIN-VALUE(*game*, game.RESULT(*state*, *a*))

if *v2* > *v* **then**

v, *move* $\leftarrow$ *v2*, *a*

return *v*, *move*

Replace IS-TERMINAL with IS-CUTOFF(*state*, *depth*).
Replace UTILITY with EVAL(*state*, *player*).

function MIN-VALUE(*game*, *state*) **returns** a (*utility*, *move*) pair

if game.IS-TERMINAL(*state*) **then return** game.UTILITY(*state*, *player*), null

v, *move* $\leftarrow +\infty$

for each *a* **in** game.ACTIONS(*state*) **do**

v2, *a2* $\leftarrow$ MAX-VALUE(*game*, game.RESULT(*state*, *a*))

if *v2* < *v* **then**

v, *move* $\leftarrow$ *v2*, *a*

return *v*, *move*

Also need to pass the depth *d* to the function calls. Depth should be incremented by 1 with each MAX or MIN call.

Alpha-beta with heuristic

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action
 $\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$
 $\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state}, -\infty, +\infty)$
 return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if $\text{game.IS-TERMINAL}(\text{state})$ **then return** $\text{game.UTILITY}(\text{state}, \text{player})$, null
 $v \leftarrow -\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 > v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\alpha \leftarrow \text{MAX}(\alpha, v)$
 if $v \geq \beta$ **then return** v, move
 return v, move

function MIN-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair
 if $\text{game.IS-TERMINAL}(\text{state})$ **then return** $\text{game.UTILITY}(\text{state}, \text{player})$, null
 $v \leftarrow +\infty$
 for each *a* **in** *game.ACTIONS*(*state*) **do**
 $v2, a2 \leftarrow \text{MAX-VALUE}(\text{game}, \text{game.RESULT}(\text{state}, a), \alpha, \beta)$
 if $v2 < v$ **then**
 $v, \text{move} \leftarrow v2, a$
 $\beta \leftarrow \text{MIN}(\beta, v)$
 if $v \leq \alpha$ **then return** v, move
 return v, move

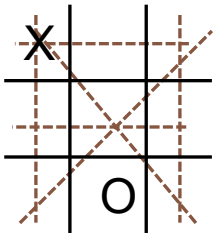
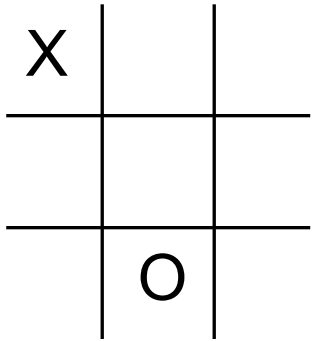
How to modify alpha-beta pseudocode with heuristic? Same as before.

Replace IS-TERMINAL with IS-CUTOFF(*state*, *depth*).
Replace UTILITY with EVAL(*state*, *player*).

Also need to pass the depth *d* to the function calls. Depth should be incremented by 1 with each MAX or MIN call.

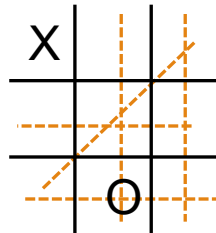
Applying h-minimax to tic-tac-toe [1]

- Let's use the following heuristic: (# win lines for X - # win lines for O)

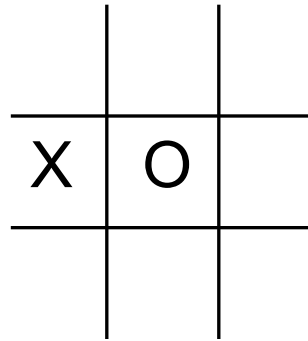


X has 6
possible win
paths

$$\text{EVAL}(s, X) = 6 - 5 = 1$$

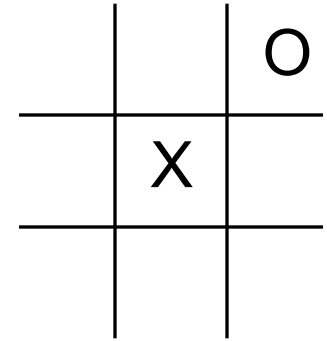


O has 5
possible win
paths



X has 4 possible wins
O has 6 possible wins

$$\text{EVAL}(s, X) = 4 - 6 = -2$$



X has 5 possible wins
O has 4 possible wins

$$\text{EVAL}(s, X) = 5 - 4 = 1$$

Applying h-minimax to tic-tac-toe [2]

- Let's use the following heuristic: (# win lines for X - # win lines for O)
- Depth-limit is set to 2.
- MAX selects the right-branch (X set to middle of board).

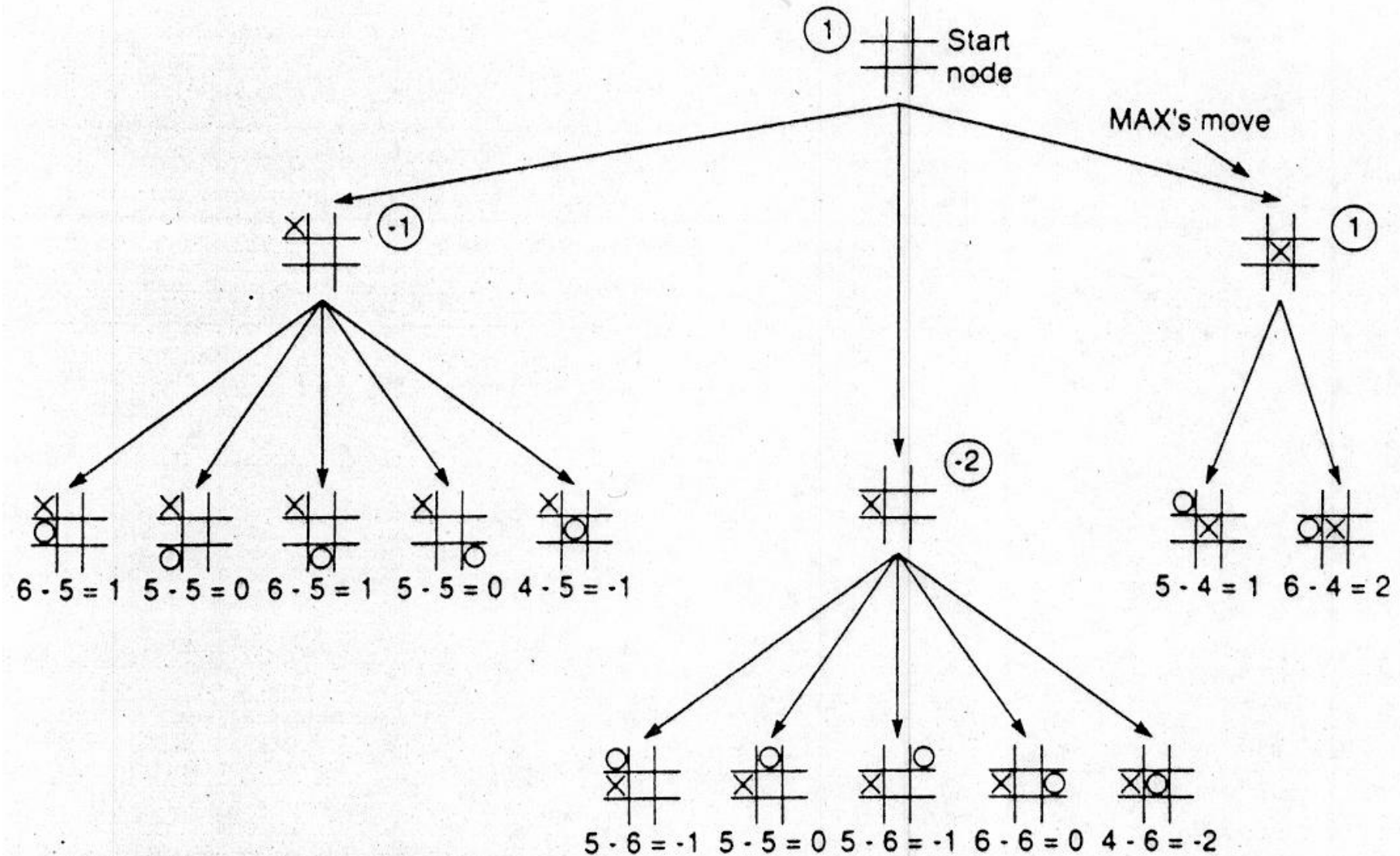


Figure 4.17 Two-ply minimax applied to the opening move of tic-tac-toe.

Applying h-minimax to tic-tac-toe [3]

- Let's use the following heuristic: ($\#$ win lines for X - $\#$ win lines for O)
- Depth-limit is set to 2.
- Real-world** movement: X moves, then O selects middle-top position.
- Now X computes h-minimax to select second move.
- X selects bottom-left position.

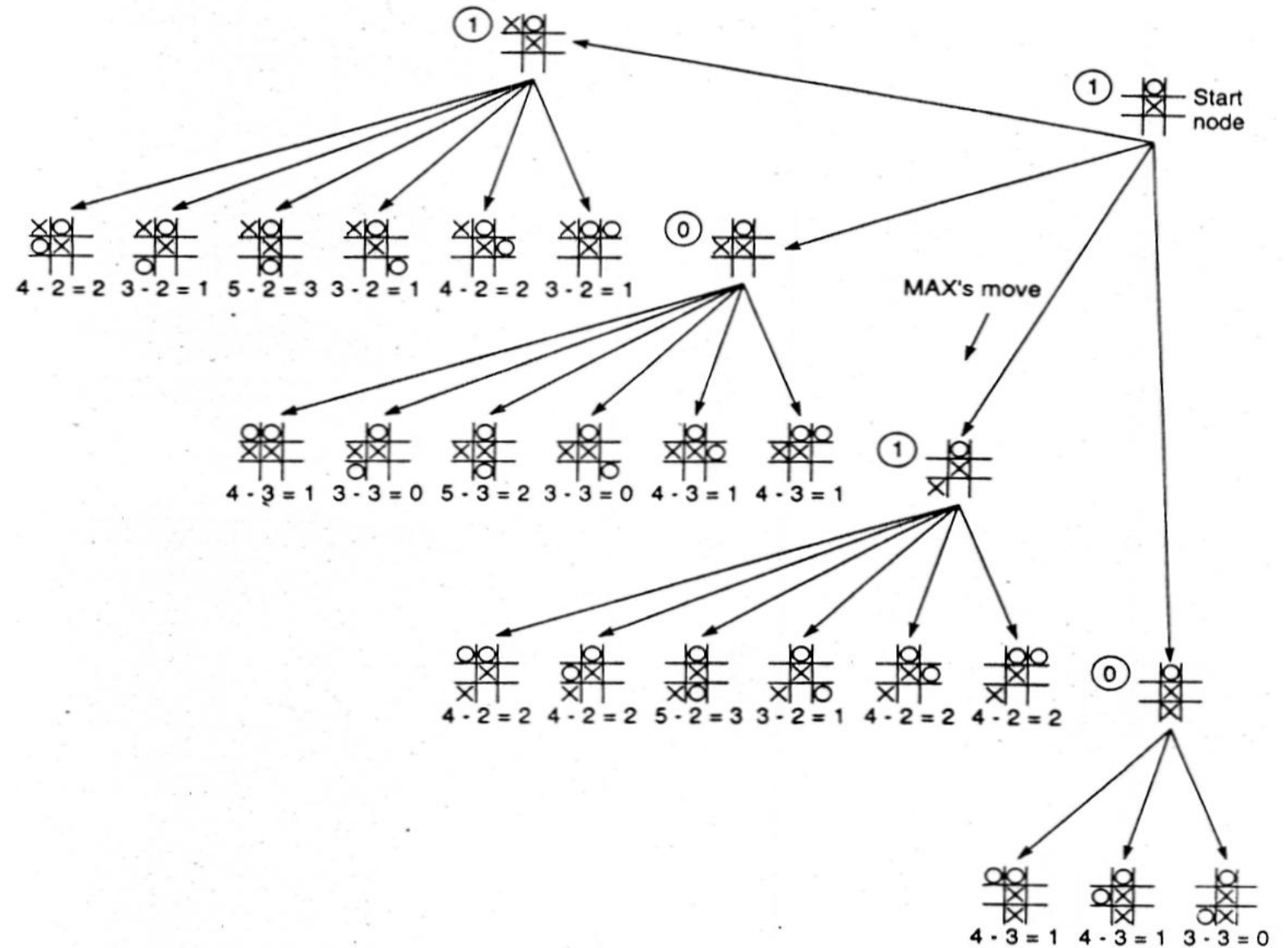


Figure 4.18 Two-ply minimax applied to X's second move of tic-tac-toe.

Applying h-minimax to tic-tac-toe [4]

- Let's use the following heuristic: (# win lines for X - # win lines for O)
- Depth-limit is set to 2.
- The game progresses in this manner. X uses h-minimax to quickly select next move.

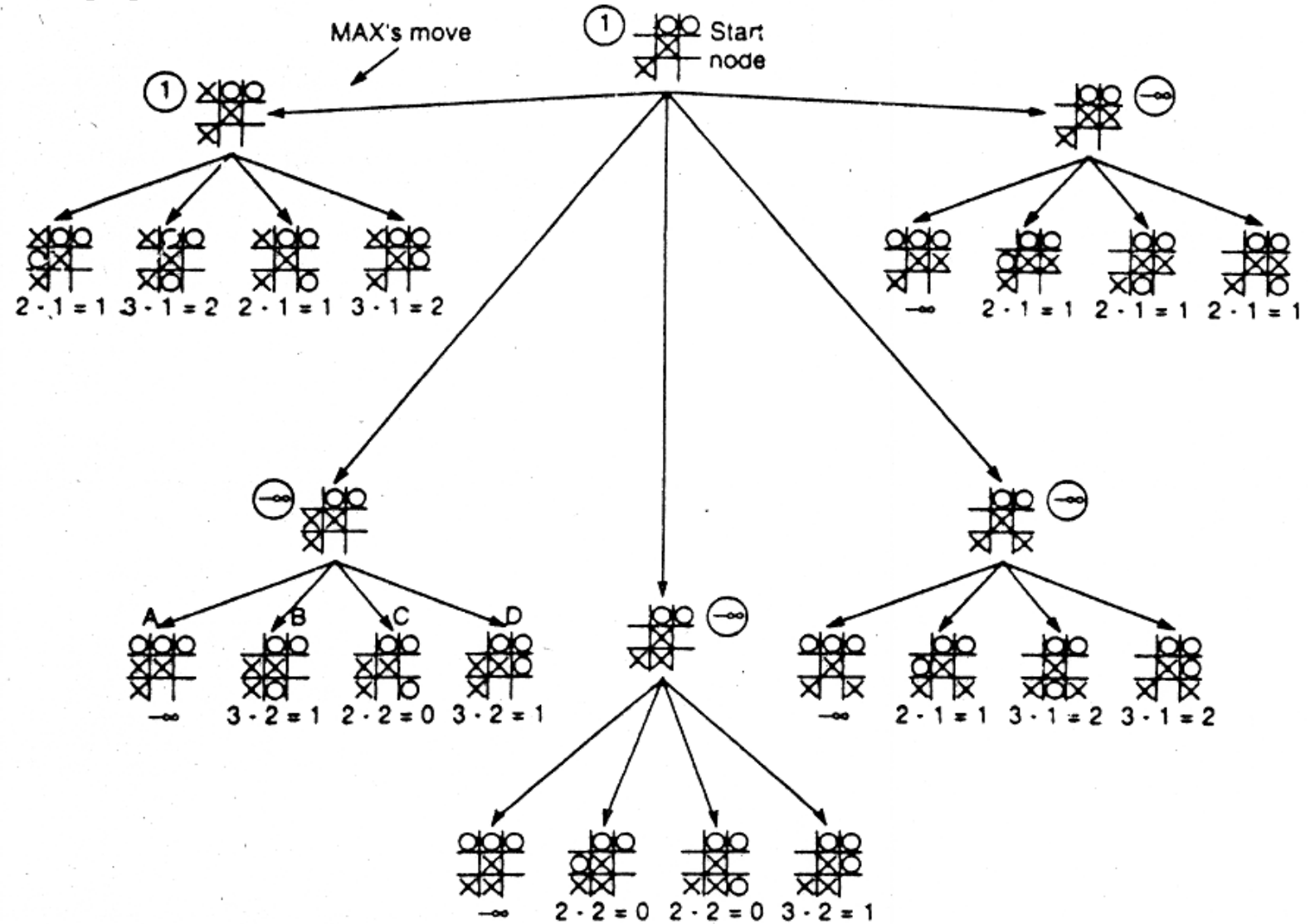


Figure 4.19 Two-ply minimax applied to X's move near end game.

Discussion on using heuristics [1]

- Is heuristic-minimax optimal? No. There can be uncertainty/error in estimating the true utility.
- Is heuristic-alpha-beta optimal? No. There can be uncertainty/error in estimating the true utility.
- So, we gain speed-up at the cost of optimality. However, with a good heuristic we can still get good outcomes (high utility terminal states).
- What if the game is timed, how to set cutoff condition?
 - **Depth-limited**: set cutoff at specific depth d such that search terminates before timer runs out.
 - Better idea: **iterative-deepening**. Grab the deepest completed search when timer runs out.
- Cutoff condition can also be restricted to specific states to improve quality of search (for those interested, look up quiescent search).
- More techniques we don't discuss here. See textbook ch 6.3.3 & 6.3.4

Discussion on using heuristics [2]

- Time complexity of alpha-beta with heuristic + cutoff at depth d ? with perfect ordering $O(b^{d/2})$
- Using some of these techniques, how fast can we make a move in chess?
- Chess has branching factor of about 35.
- Using minimax with heuristic + cutoff test with quiescence + a machine that can evaluate 1 million nodes per second.
 - With time-limit 1 minute, can search up to 5 ply (depth 5) in the game-tree.
 - Can be defeated by average human chess player.
- Using alpha-beta with heuristic + cutoff test with quiescence + a machine that can evaluate 1 million nodes per second + large lookup table
 - With time-limit 1 minute, can search up to 14 ply (depth 14) in the game-tree.
 - Expert level play.
- [Stockfish](#): alpha-beta with fine-tuned heuristic (reinforcement learning self-play) + cutoff test with quiescence + lookup table for known states + database of opening moves + lots of compute resources (can use up to 1024 CPU threads and transposition table of 32TB). Can reach 30 ply (depth 30). Grandmaster level play.
- See also [AlphaZero](#)

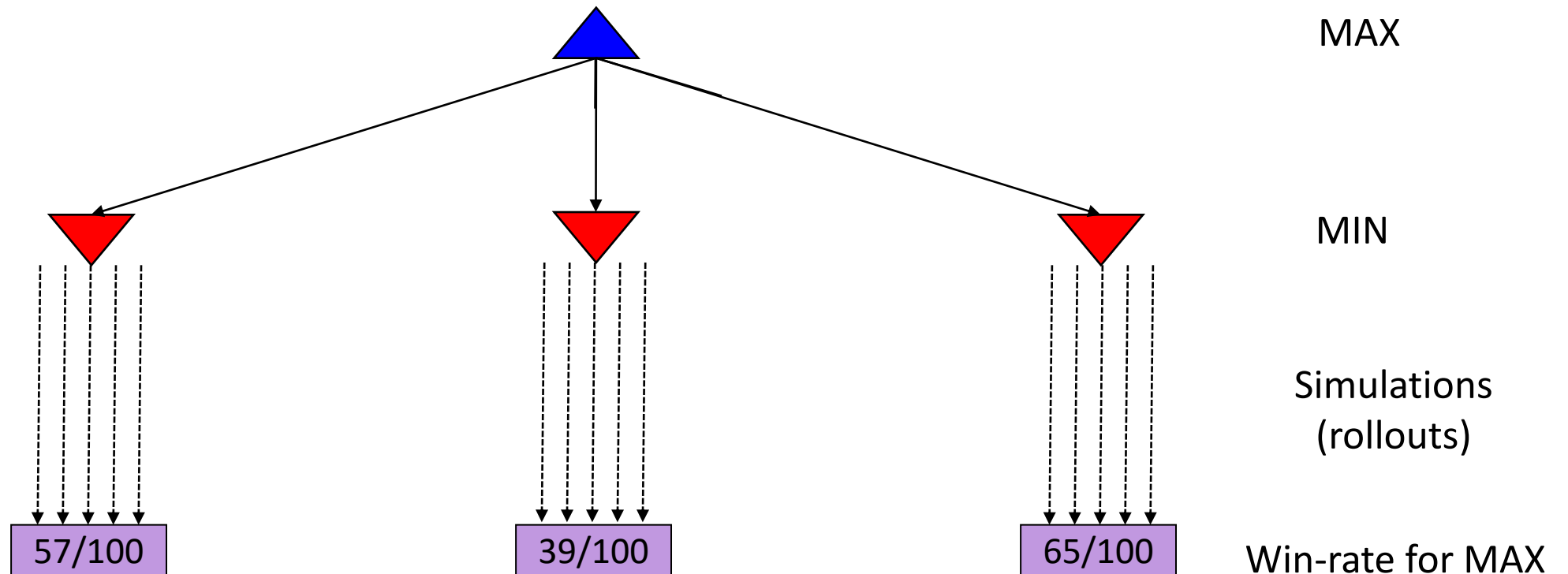
Limited resources – speeding up the
game search with
simulations (playouts)

Monte Carlo tree search [1]

- Time complexity of alpha-beta with heuristic + cutoff at depth d ? with perfect ordering $O(b^{d/2})$
- For some games like Go:
 - large branching factor (Go starts at 361), this is still too many nodes to explore.
 - difficult to define a good heuristic evaluation function.
- **Monte Carlo tree search (MCTS)** is suitable strategy for games like Go with large branching factor or when good heuristic is difficult to define.
- Instead of using heuristics, the value of a state is *estimated* as the average utility over a number of **simulations** starting from that state.
- A **simulation (rollout, playout)** is a **computational playout** of the game starting from some current state. The simulation will take player turns until the game terminates.
- This requires a **playout policy** that determines how the players in the simulation will act.
- **Random** playout can work:
 - Start the game from some current state. Simulate computationally the turns of the two players until end-game state.
 - The actions taken by the players are uniformly random.
 - This can work for most games, but it can be inefficient in terms of gathering good statistics. There are other options for the simulation playout using ideas from reinforcement learning (not covered here).

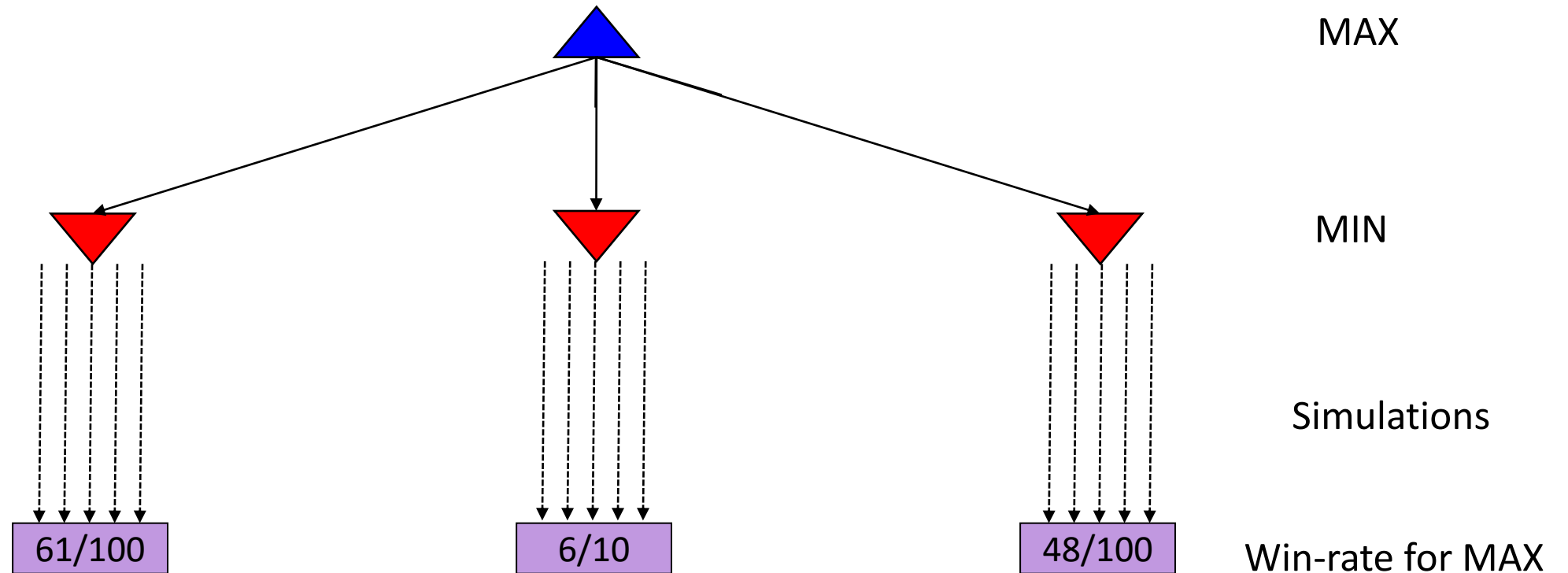
Equal (uniform) selection policy

- Given a budget of N simulations, equally distribute these rollouts to each child, record fraction of wins.
- This is an equal selective policy that assigns the same number of simulations to each child.
- MAX then picks the move that gives the best win-rate for MAX.



Exploitive and explorative selection policy

- Even better idea: use a selection policy to distribute the rollouts
 - Allocate simulations to **uncertain** nodes (exploration).
 - Allocate simulations to **promising** nodes (exploitation).
 - For example, if we had a budget of N simulations, we can do several iterations of exploration, then several iterations of exploitation.



MCTS with UCB1 selection policy

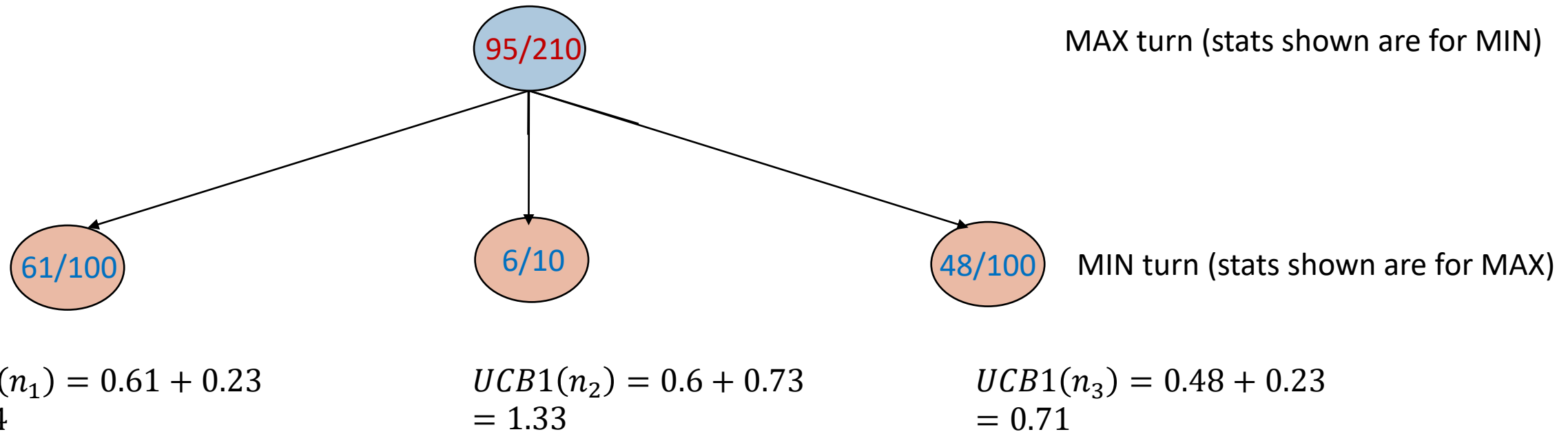
- MCTS makes use of UCB (Upper-confidence bound) idea. For those interested this is from the multi-armed bandits problem.
- Specifically, MCTS uses UCB1 formula to rank game-tree nodes. Combines “promising” and “uncertain”:

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$

- $U(n)$ = total utility of rollouts (simulations) for node n (e.g., # wins)
- $N(n)$ = number of rollouts (simulations) from node n
- **Exploitation term**: first term is the **estimated** average utility of n . A node with higher avg utility is more **promising**.
- **Exploration term**: second term with the square root. Is large when n has been simulated a few times relative to its siblings (summarized by simulations at the parent); so we are **uncertain** about its **estimated** average utility (first term).
- C is a constant to trade-off exploitation vs exploration. Theoretical argument that this can be $C = \sqrt{2}$.

UCB1 calculation example

- We want to use the UCB1 formula to select the next child to assign a simulation. To simplify, we'll use $C = 1$.
- We'd select the middle child because it has highest ucb1 value.



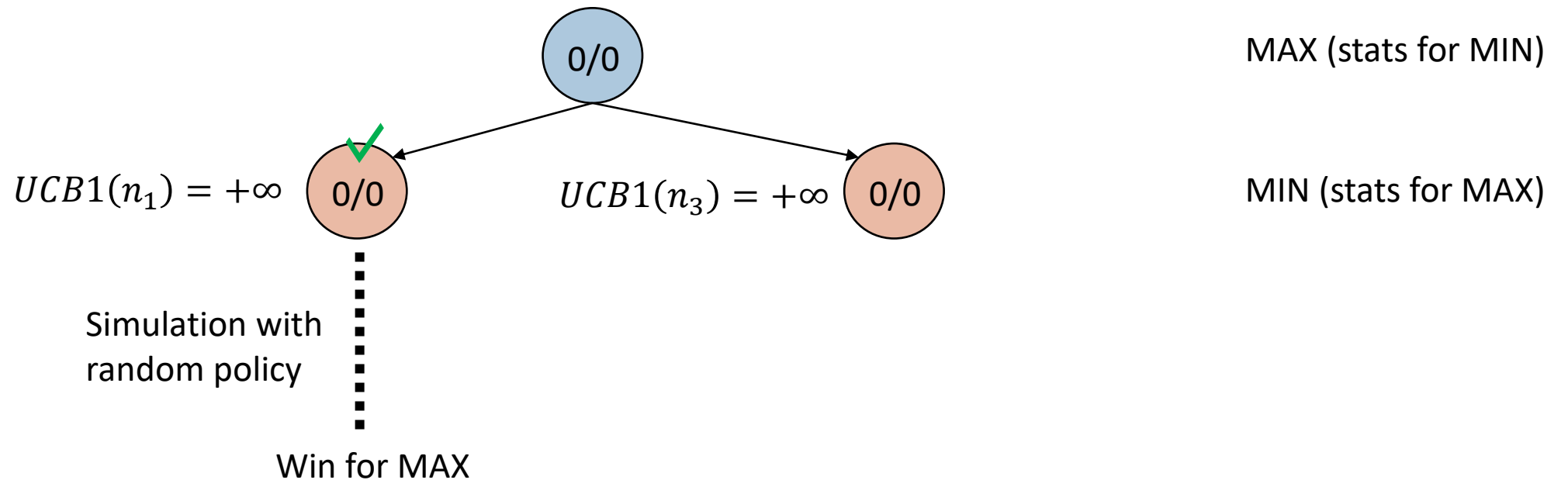
MCTS Overview

- At the start, MCTS has the single root node in the game-tree for the given game state.
- In each iteration, MCTS **grows** the game-tree by doing the following three steps:
 - **Selection:** Given the current search tree's root node, recursively apply UCB1 to choose a path down to a **leaf** node.
 - **Simulation:** run a simulation from the leaf node.
 - **Back-propagation:** Update the stats back up to the root.
- When time runs out, choose the action from the root leading to the child with highest number of simulations.
- Why not choose child with highest average utility?
 - Nodes with more playouts have less uncertainty in their estimates. i.e. node with 65/100 wins has less uncertainty than node with 2/3 wins.
 - With UCB1 selection policy the child with highest number of simulations is *almost always* the one with highest average utility. This is because UCB1 selection favors win percentage more as the number of playouts increase.
- See [link](#) for more description of MCTS operation.
- Used in chess bot [Komodo](#). Used in Go bot [AlphaGo](#).

MCTS Full Example; iteration 1

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$

- The root will represent the current game state. Let us assume that all game states have two actions. We'll use $C = 1$ to simplify calculations.
- **Special case:** nodes with no stats have $UCB1 = +\infty$. We consider these **leaf nodes** (except for root).
- At start, the tree will just be the root node and its children (actions available to MAX). Here let's assume MAX and MIN have 2 actions.
- All children of root have no stats. They have same UCB1 value, so just select uniformly at random. We'll select the left child.
- Run **simulation** at left child. Let's say it is win for MAX. **Generate** children nodes for left child. Then **backpropagate** stats to root.



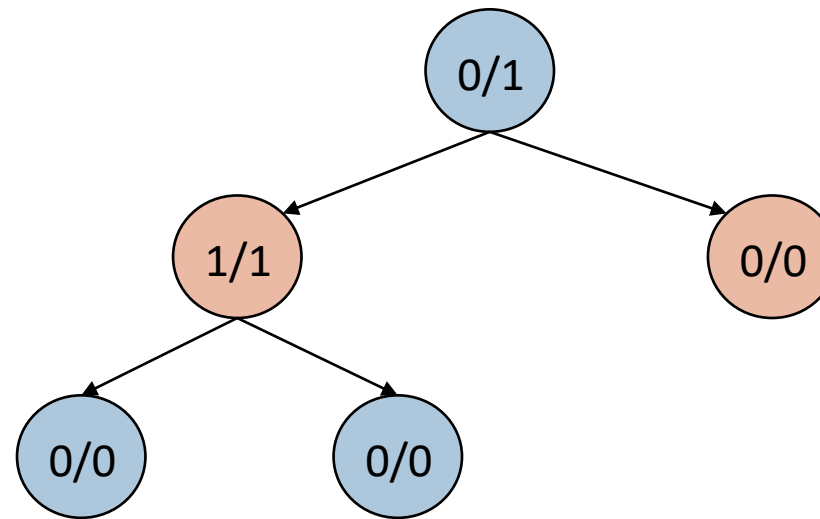
MCTS Full Example; iteration 1

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$

- Run **simulation** at left child. Let's say it is win for MAX. **Generate** children nodes for left child. Then **backpropagate** stats to root.
- We just finished iteration 1.

After simulation:

- Update stat of node
- Generate children node
- Backpropagate to root



MAX (stats for MIN)

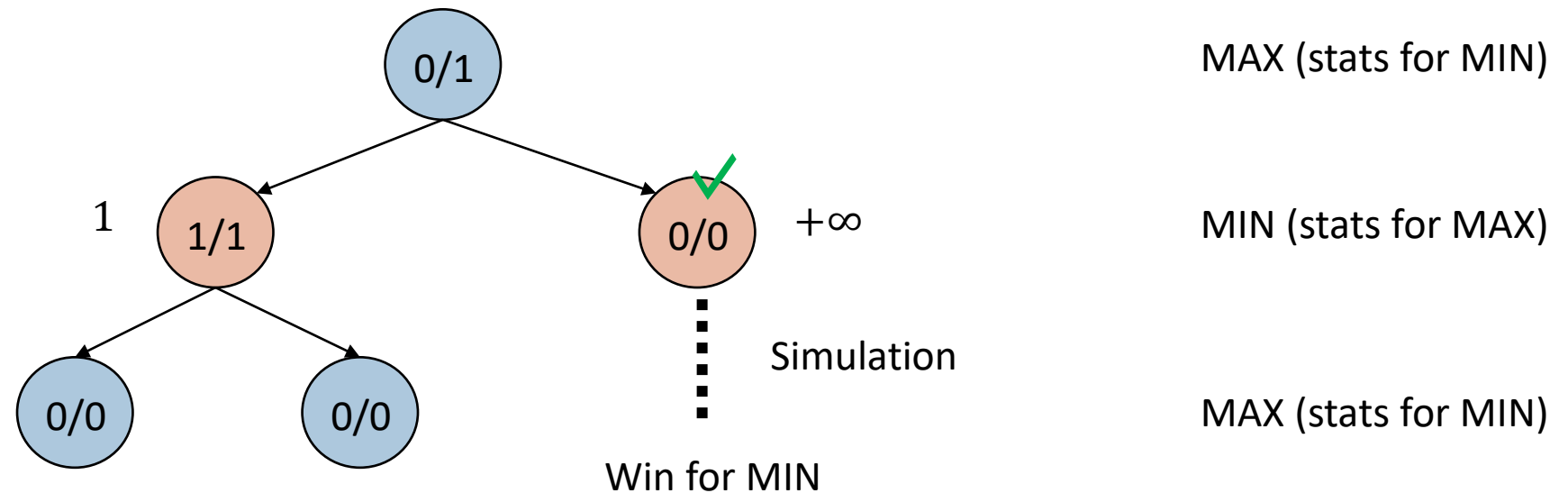
MIN (stats for MAX)

MAX (stats for MIN)

MCTS Full Example; iteration 2

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$

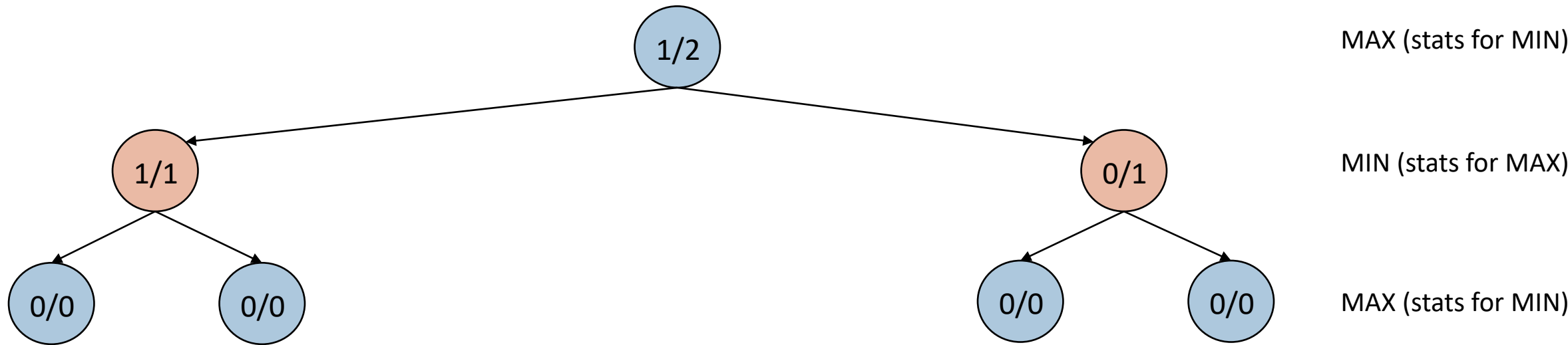
- At iteration 2, we start at the root and use UCB1 to move down the tree until we hit a leaf node (node with $UBC1 = +\infty$). We select children with highest UCB1.
- Run **simulation** at selected leaf node. **Generate** children nodes at simulation node. Then **backpropagate** stats to root.



MCTS Full Example; iteration 2

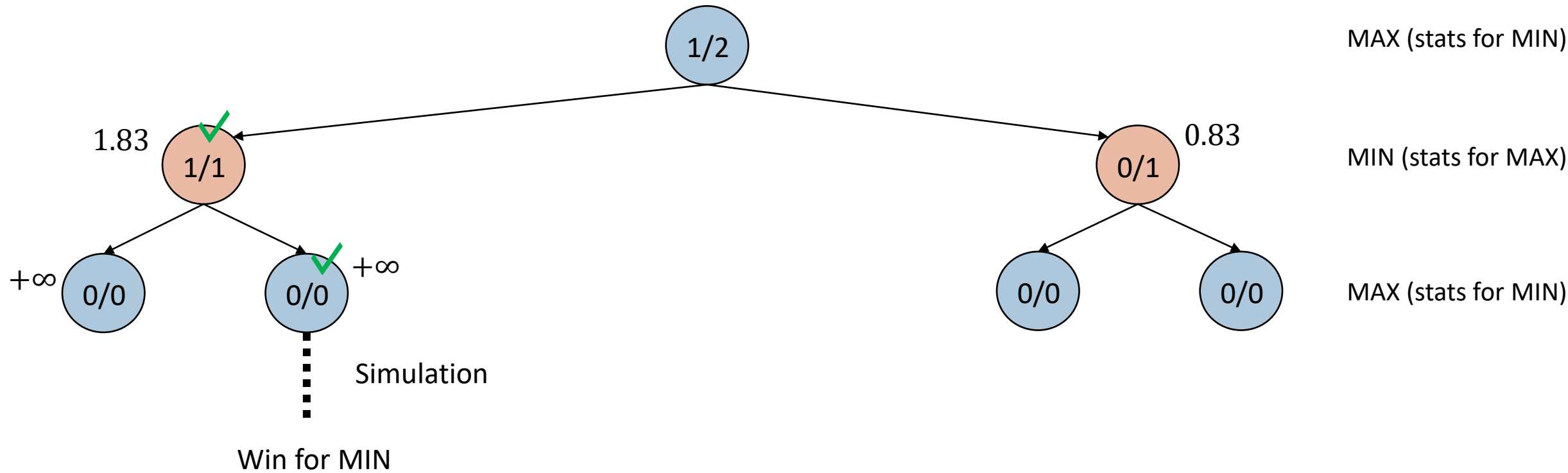
$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$

- At iteration 2, we start at the root and use UCB1 to move down the tree until we hit a leaf node (node with $UBC1 = +\infty$). We select children with highest UCB1.
- Run **simulation** at selected leaf node. **Generate** children nodes at simulation node. Then **backpropagate** stats to root.
- We just finished iteration 2.



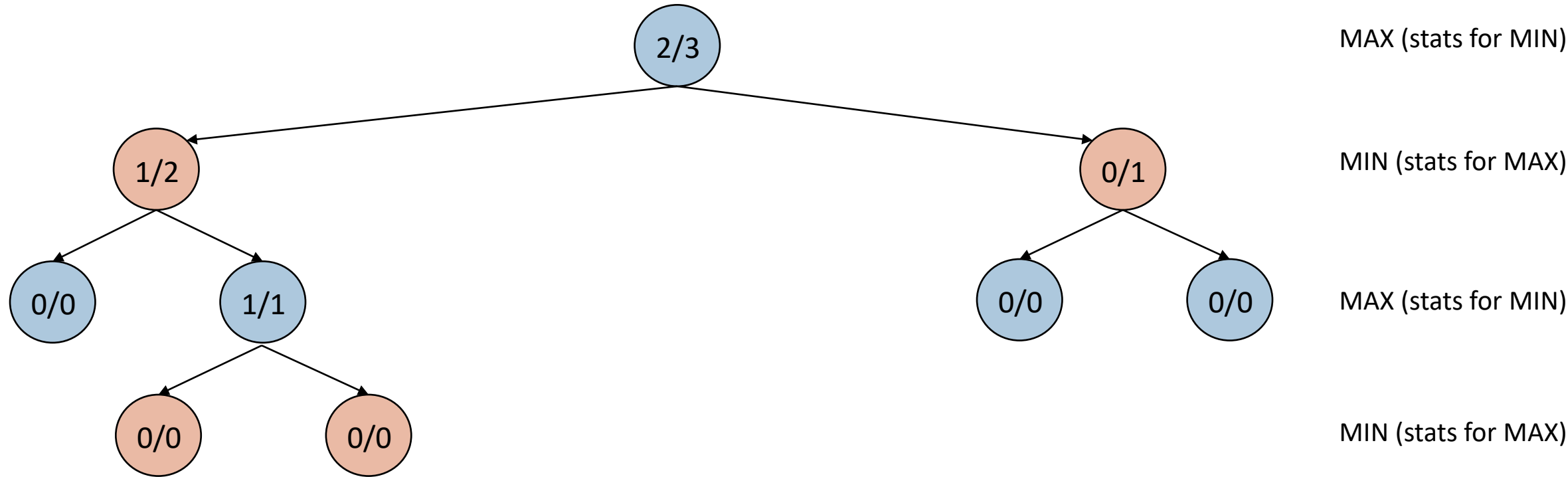
MCTS Full Example; iteration 3

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$



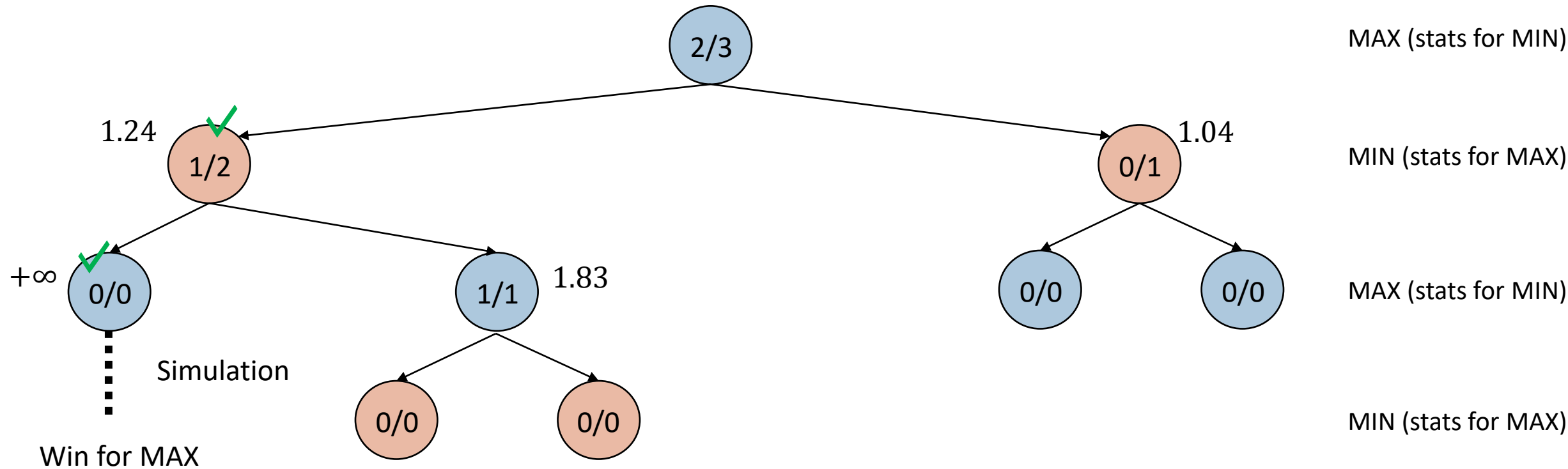
MCTS Full Example; iteration 3

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$



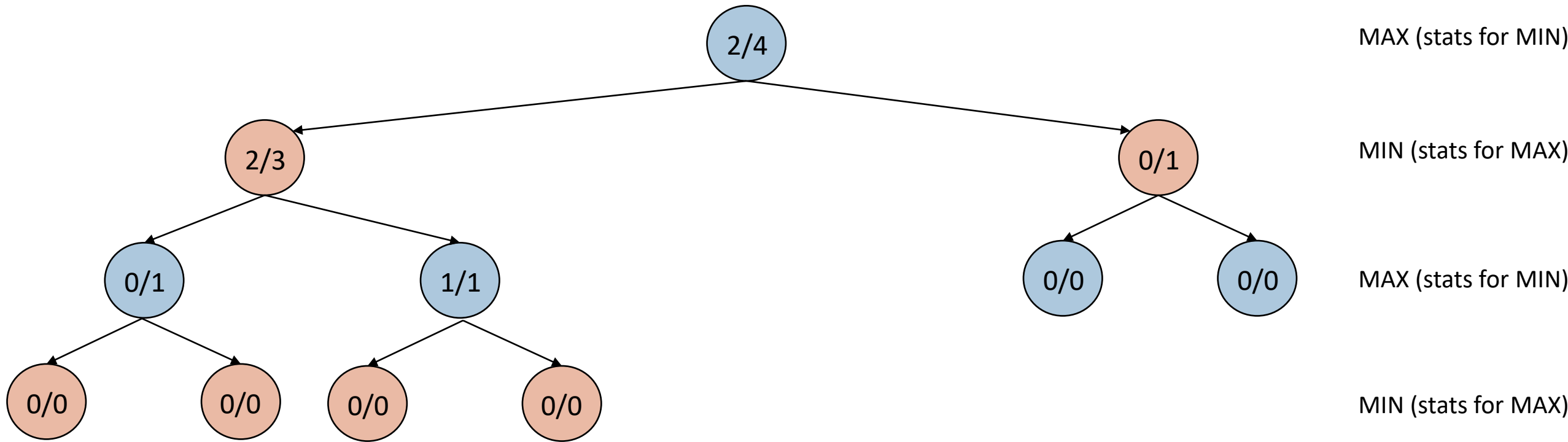
MCTS Full Example; iteration 4

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$



MCTS Full Example; iteration 4

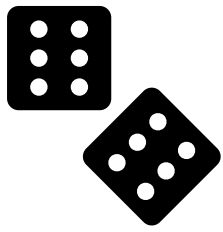
$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\ln N(\text{PARENT}(n))}{N(n)}}$$



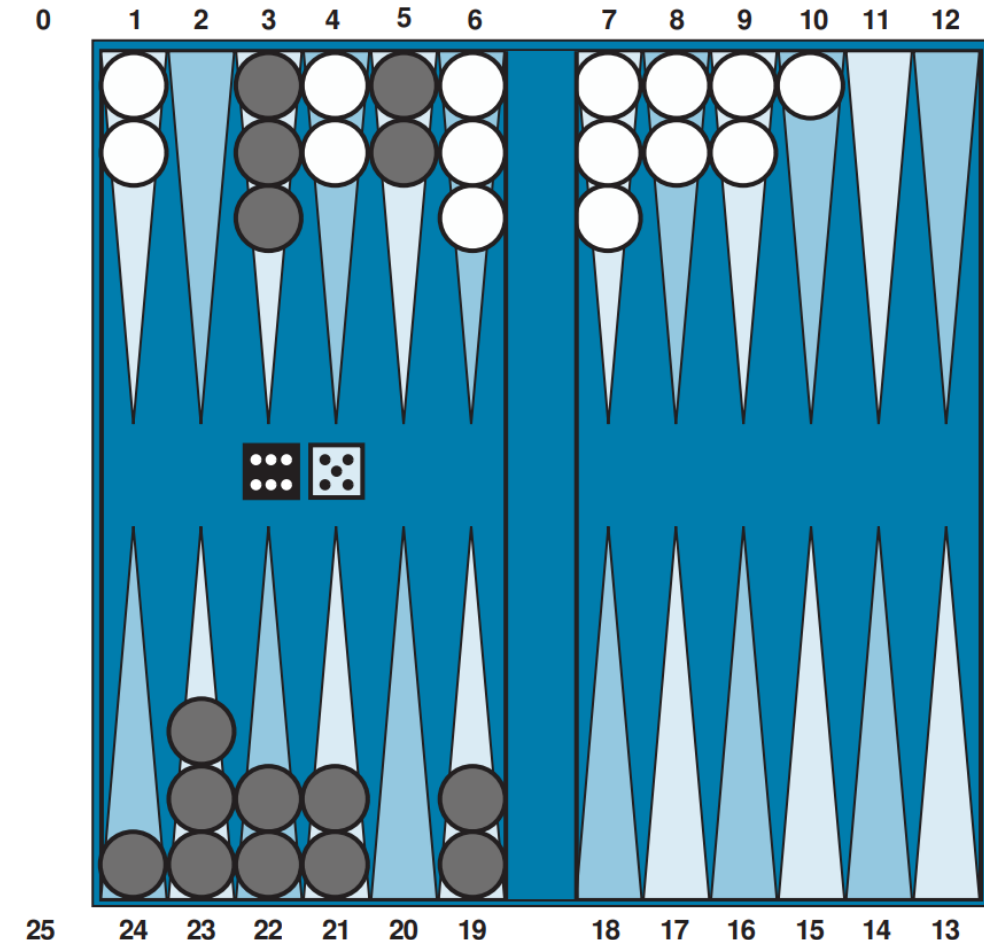
- You keep proceeding to grow the tree with each iteration. Doing Selection of leaf, simulation of leaf, then backpropagate stats.
- When time runs out or until a specific max-iteration is reached, choose the action from the root leading to the child with highest number of playouts.
- So, for current state of tree above, we would select the left child.

Stochastic Games – games with element of randomness

Stochastic Games – intro



- Stochastic games are games that have an element of **randomness**.
- A common source of game randomness is the use of dice in games like backgammon.
- In such games, the game-tree needs to account for this randomness via **chance nodes**.
- So, in stochastic games, we have MAX-nodes, MIN-nodes, and chance-nodes.
- For example, in backgammon, the player first rolls two 6-sided dice, then depending on the outcome of the throw, the player can make certain moves (actions).

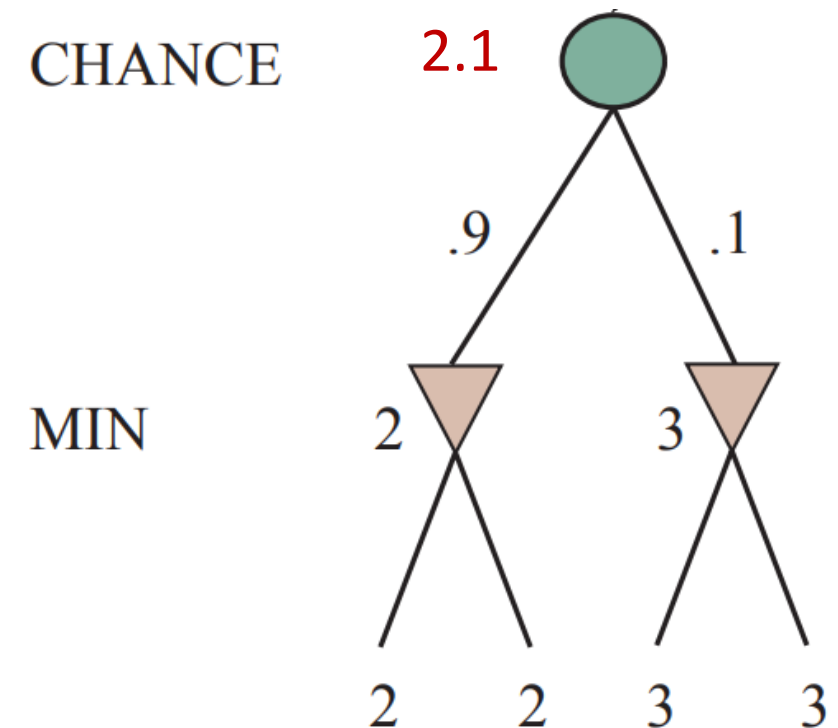


Game-tree with chance nodes

- Now that we've discussed what the stochastic game-tree looks like, how does MAX select the best move?
- Recall from minimax that the utility value of a MAX and MIN node are the max and min of their child nodes, respectively.
- Since probabilities are involved, we can select the move with best **expected value**.
- What about the utility value of a chance node?
- It is the weighted sum of its children, weighed by the probability of each child.

$$\sum_{c \in \text{children}} \text{Prob}(c) \text{Utility}(c)$$

- For example shown, what is the value of the chance node?
 $(0.9 * 2) + (0.1 * 3) = 2.1$

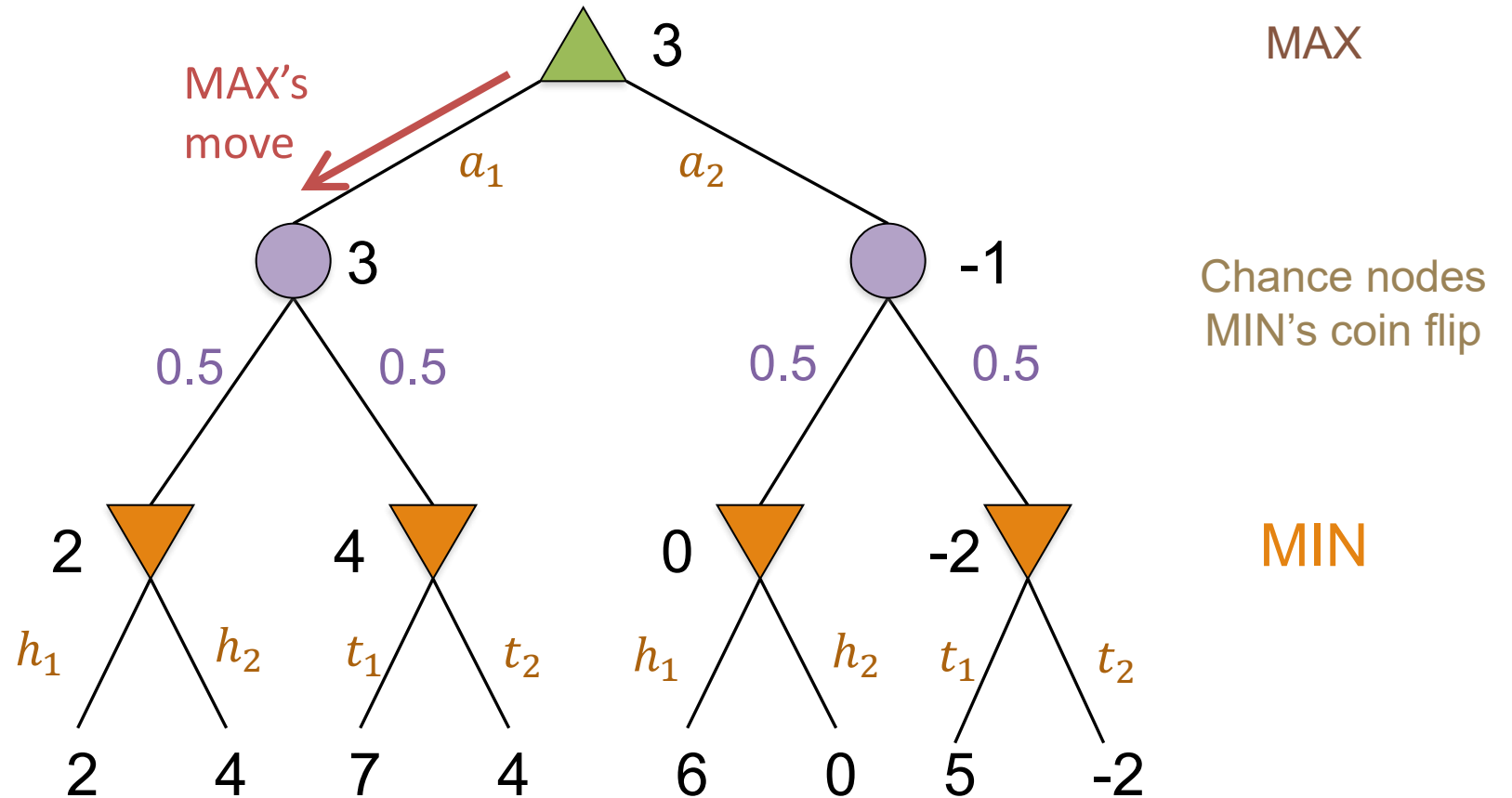


Game-tree with chance-nodes

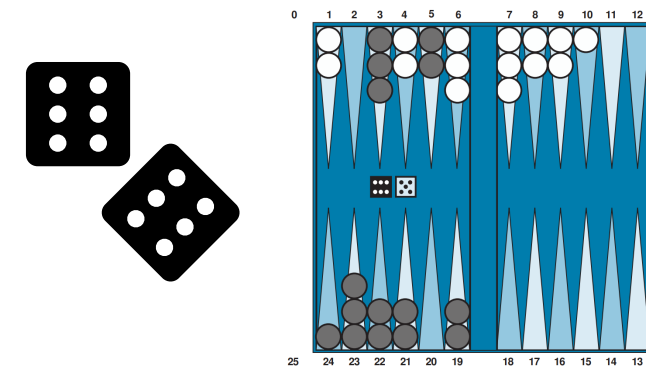
Consider a simple 2-turn game with a coin. MAX has already thrown the coin, and already determined its outcome, so the root node-state is already determined for MAX and corresponding actions are determined. Next turn, MIN will throw a coin, and depending on outcome it will have different actions available. MAX is using the following game-tree to model this behavior and select the best action.

Chance node with two branches for the two outcomes of the coin flip. Branches labelled by the probability of the outcome.

The utility value of chance node is the expected utility.



Game-tree with two 6-sided dice in backgammon [1]



- Now consider a game with dice like backgammon.
- Suppose it is MAX's turn, how do we compute the best move?
- First, MAX will throw dice, and will know for certain the set of legal moves available.
- But, to search the game-tree we need to account for **future** possible dice throw outcomes by both MAX and MIN.
- There are 36 possible outcomes for two 6-sided dice.
- Assuming the dice are **fair**, then the **probability** of each outcome is $1/36$.
- For games like backgammon, 5-6 and 6-5 both lead to the same set of legal moves, thus there are 21 unique outcomes.
 - The six doubles (1-1, 2-2, ..., 6-6) each with probability $1/36$
 - The 15 unique permutes (e.g. 3-2 and 2-3) each with probability $2/36 = 1/18$
 - Probability of all outcomes at CHANCE levels need to sum to 1

Game-tree with two 6-sided dice in backgammon [2]

MAX

The root is a MAX-node. MAX has already made a roll and knows the legal moves available.

CHANCE
MIN

Chance node with branches for the possible rolls that MIN can make. There is a chance node for each action made by MAX above.

MIN

Depending on MIN's roll outcome, MIN node has certain actions available.

CHANCE
MAX

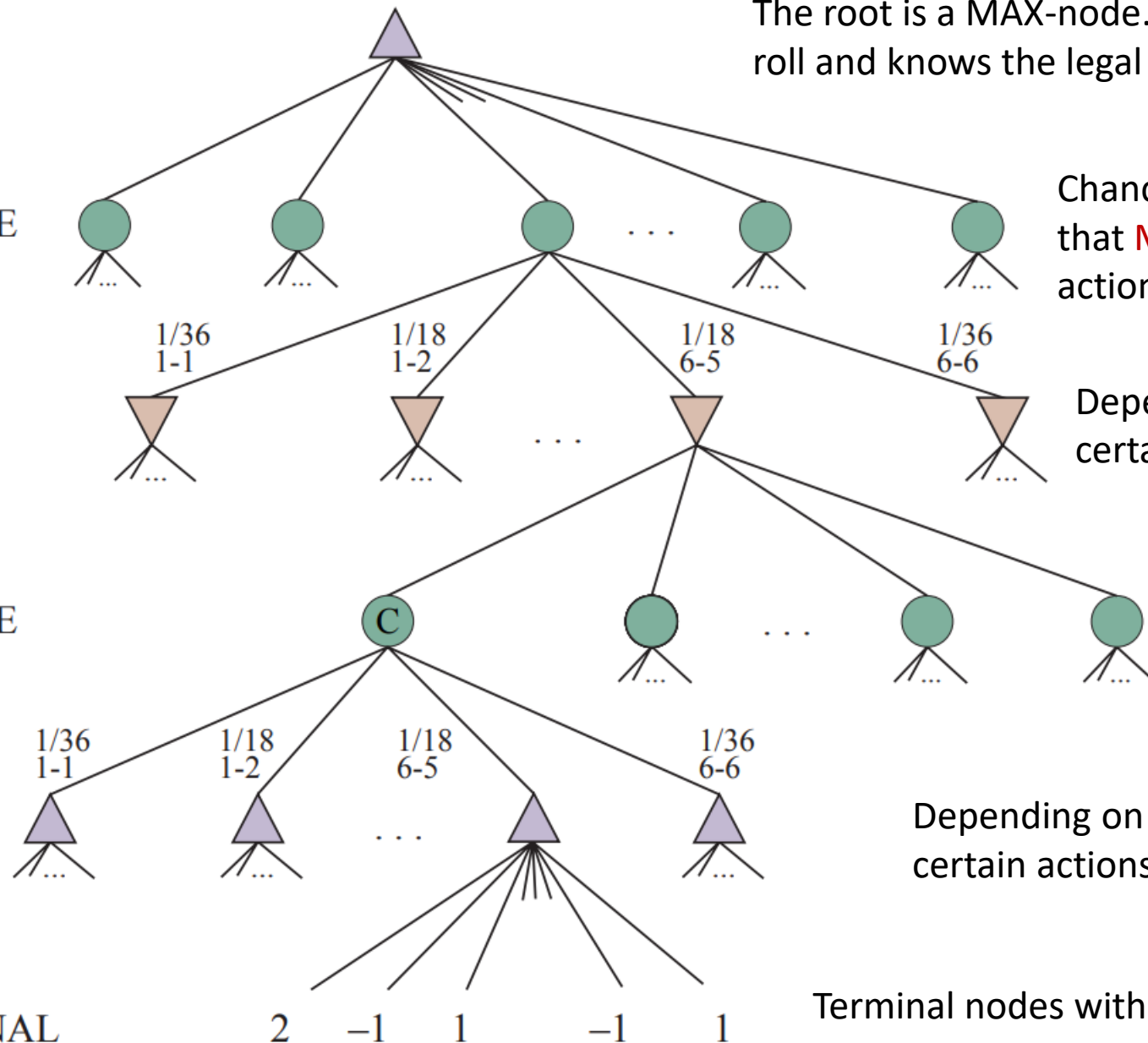
Chance nodes for MAX's roll 2 turns later.

MAX

Depending on MAX's roll outcome, MAX node has certain actions available.

TERMINAL

Terminal nodes with utility values for MAX



Expectiminimax

- Thus, we can compute utility of MIN, MAX, and Chance nodes which gives us the **expectiminimax** algorithm.

$$\text{EXPECTIMINIMAX}(s) = \begin{cases} \text{UTILITY}(s, \text{MAX}) & \text{if } \text{TERMINAL-TEST}(s) \\ \max_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{TO-MOVE}(s) = \text{MAX} \\ \min_a \text{EXPECTIMINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{TO-MOVE}(s) = \text{MIN} \\ \sum_r P(r) \text{EXPECTIMINIMAX}(\text{RESULT}(s, r)) & \text{if } \text{TO-MOVE}(s) = \text{CHANCE} \end{cases}$$

- The minimax pseudocode can be modified accordingly to turn it into expectiminimax. E.g. add a function for the chance-node.

Properties Expectiminimax

- Time complexity:
 - $O(d)$ is number of turns until game ends
 - Is it same as minimax $O(b^d)$? No, need to account for chance nodes.
 - If there are $O(n)$ branches for a chance node, then expectiminimax is $O(b^d n^d)$
- How to speed up expectiminimax search:
 - Can cutoff the search and use a heuristic function or monte-carlo simulations.
 - Can use an adapted form of alpha-beta pruning. For chance nodes, alpha-beta are bounds on the expected utility. (won't be discussed in this course)

Summary

Summary

- Game playing can be effectively modeled as a search problem where multiple agents compete against each other.
- A game-tree represents alternating agent actions.
- Minimax is a procedure which chooses moves by assuming that agents will always choose the move which is best for them (optimal).
- Alpha-beta pruning is a procedure which can prune large parts of the search tree and allows the search to go deeper.
- Heuristic evaluation functions estimate the quality of a given state configuration. Using heuristics, we can cutoff the search early and evaluate non-terminal nodes.
- An alternative to using heuristics is to gather simulated playout statistics for each state. This is what Monte Carlo tree search does. It is suitable for games with a large branching factor or when a good heuristic is difficult to define.
- Game search techniques can be adapted to stochastic games via expected value.
- For many well-known games, computer algorithms based on heuristic search match or out-perform human world experts.